THESIS

Qudrat Ullah

Debrecen
2026

UNIVERSITY OF DEBRECEN
FACULTY OF INFORMATICS
DEPARTMENT OF DATA SCIENCE AND VISUALIZATION

THESIS

A Hybrid Multimodal Lab Assistant: Blood Image Classification Combined with Domain-Expert LLM Reasoning and RAG

Supervisor:
Prof. Dr. András Hajdu
University professor, dean

Candidate:
Qudrat Ullah
Computer Science BSc

DEBRECEN
2026

Contents

Abstract **3**

1 Introduction **4**

1.1 Aims and research questions 4

1.2 Research methods 5

1.3 Structure of the thesis 5

2 Theoretical Background **7**

2.1 Large language models and their limits in medicine 7

2.2 Building domain experts from general LLMs 7

2.3 Tool use and the ReAct agent paradigm 8

2.4 Evaluating a domain expert 9

2.5 Automated peripheral blood-smear analysis 9

2.6 Uncertainty and explainability 10

2.7 Trustworthy AI in medical decision support 11

3 Data and Knowledge Sources **12**

3.1 Hematology textbook corpus 12

3.2 Synthetic question–answer dataset 13

3.3 External multiple-choice questions (MedMCQA) 14

3.4 Cell detection data: TXL-PBC 14

3.5 White-cell classification data: PBC 15

4 System Architecture and Design **16**

4.1 Architectural overview 16

4.2 Stage 1: cell detection with YOLOv8 17

4.3 Stage 2: white-cell classification with uncertainty and saliency 18

4.4 Stage 3: domain-expert reasoning — knowledge base and retrieval (RAG) . . . 18

4.5 Stage 3, backend A: GPT-4o with tools (ReAct agent) 19

4.6 Stage 3, backend B: a fine-tuned open-weight model 21

4.7 Safety logic: the expert-review rule 21

4.8 Software implementation and user interface 22

5 Research Methodology **24**
5.1 Training the image-analysis models 24
5.2 Evaluation of the image-analysis stages 25
5.3 Training the domain-expert model 25
5.4 Evaluation protocol for the domain expert 26

6 Results and Discussion **29**
6.1 Stage 1: detection results 29
6.2 Stage 2: classification and uncertainty results 31
6.3 Quantitative evaluation of the domain expert 36
6.4 Retrieval quality of the knowledge base 39
6.5 Qualitative analysis: citations and errors 40
6.6 End-to-end case study: an over-interpreted differential 42
6.7 Discussion of the research questions 44
6.8 Limitations and threats to validity 45

7 Conclusion **47**
7.1 What was not achieved 48
7.2 Future work 48

8 Bibliography **50**

9 Appendix **54**
A. Reproducing the results 54
B. Main configuration values 55
C. Output of the domain-expert agent (abridged) 56
D. Declaration on the use of generative AI 56

Acknowledgment **58**

Abstract

General-purpose large language models such as ChatGPT answer medical questions fluently, but in a specialised field they can state unsupported facts, cannot show where an answer comes from, and know nothing about the case at hand. This thesis addresses the topic of developing a domain expert with such a model by building a hybrid multimodal lab assistant for peripheral blood smears. Two image-analysis stages supply case evidence: a YOLOv8s detector locates white cells, red cells and platelets (mAP@0.50 = 0.985 on TXL-PBC), and an EfficientNet-B0 classifier assigns each white cell to one of the eight PBC classes — six white-cell types plus erythroblasts and platelets (98.60 % accuracy on a held-out PBC test set). Every classified cell carries a Monte Carlo Dropout uncertainty estimate, which identifies misclassified cells with an AUROC of 0.97 on the PBC test set, and a Grad-CAM saliency map. The domain expert interprets these findings in two forms: a GPT-4o agent that retrieves passages from 1,049 hematology textbook chunks (retrieval-augmented generation) and uses tools for reference ranges, cell counts and uncertainty; and an open-weight Llama-3.1-8B model fine-tuned with QLoRA on 1,898 textbook-grounded question–answer pairs. Fine-tuning made the open model's answers much closer to textbook reference answers (ROUGE-L 0.170 $\rightarrow$ 0.455) and stopped it from attributing its answers to textbooks it had never been given, but did not improve its clinical correctness or its accuracy on external exam questions. Given retrieved textbook passages, GPT-4o became more faithful and more correct (LLM-judge scores 4.02 $\rightarrow$ 4.24 and 4.67 $\rightarrow$ 4.83 on a 1–5 scale) and cited only passages it had received. Deterministic rules, independent of the language model, force a human review of uncertain cases; real failures — an over-interpreted two-cell differential and citations of passages the agent had never retrieved — led to additional rules. The results suggest a layered design for LLM domain experts: fine-tuning for behaviour, retrieval for knowledge, tools for case data, and explicit rules for safety.

Keywords: large language models, domain expert, retrieval-augmented generation, QLoRA, blood smear, YOLOv8, EfficientNet, uncertainty estimation, clinical decision support

1 Introduction

This thesis was written on the topic *Development of a domain expert using a large language model like ChatGPT*. Instruction-tuned large language models (LLMs) answer many medical exam questions above the pass mark [1], so it is tempting to use such a model directly as an "expert". In a clinical field, however, a general-purpose model has three weaknesses: it can produce fluent but unsupported claims (hallucination [2]); it offers no provenance, and may even invent references that do not exist [3]; and it has no access to the case — here, the cells on a blood-smear image — so its answers are generic. In this thesis, a *domain expert* is therefore an LLM-based system that answers within one field, grounds its statements in curated domain literature and cites it, inspects case-specific evidence through tools, and signals when a human expert must take over.

Choice of application. The topic is addressed through a concrete system: a *hybrid multimodal lab assistant* for peripheral blood smears. The blood smear is one of the most informative tests in hematology: a stained drop of blood is examined under the microscope, and the proportions of the white-cell types (the *differential*) can reveal infection, leukaemia or bone-marrow disease. Manual review is slow and observer-dependent, which has motivated much work on automated cell classification [4, 5]; but such systems stop at a label per cell. Turning the labels into a clinical interpretation requires domain knowledge. Hematology is therefore a natural test bed for an LLM domain expert, because it combines visual, case-specific evidence, which image models can extract, with textbook knowledge, which the expert must supply.

Theoretical and practical importance. Theoretically, the thesis compares the main ways of turning a general LLM into a domain expert — retrieval-augmented generation (RAG), tool use and parameter-efficient fine-tuning — on the same domain, and asks what each of them actually contributes. Practically, it shows how such an expert can be connected to image analysis in a way that is inspectable and safe: every automated finding carries an uncertainty estimate, every claim can be traced to a retrieved source, and the decision to involve a human is taken by explicit rules rather than by the model. It also examines whether an open model that can run on local hardware can replace a commercial API, which matters wherever patient data may not leave the institution.

1.1 Aims and research questions

The aim is to build and evaluate the lab assistant and, through it, to answer four research questions:

RQ1 Can standard deep-learning models (YOLOv8, EfficientNet-B0) detect and classify blood

cells accurately and attach a useful uncertainty estimate and a saliency map to every classified white cell?

RQ2 How can a general-purpose LLM be turned into a hematology domain expert with retrieval-augmented generation and domain-specific tools, and what does the resulting assistant produce on real smear images?

RQ3 Does QLoRA fine-tuning of an open-weight model (Llama-3.1-8B-Instruct) on textbook-grounded question–answer pairs improve its hematology answers compared with the same model without fine-tuning?

RQ4 Can the combined system make its reasoning inspectable and hand difficult cases to a human through deterministic rules — and where does this fail?

1.2 Research methods

The work combines system development with quantitative evaluation. The image-analysis stages were trained on public benchmark datasets (TXL-PBC for detection, PBC for classification) and evaluated on held-out test sets with standard metrics, complemented by a calibration and error-detection analysis of the uncertainty estimate. The domain expert was built from a knowledge base of 1,049 textbook passages, a GPT-4o agent with five hematology tools, and a Llama-3.1-8B model fine-tuned with QLoRA on synthetic question–answer pairs generated from the same textbooks. The fine-tuned model was compared with its base version and with GPT-4o, with and without retrieved passages, on held-out questions (similarity to reference answers and ratings by an LLM judge) and on external exam questions, using paired statistical tests. Finally, a complete run of the assistant was analysed as a case study. All code, notebooks, configuration and results are part of the submitted software artefact.

The main findings, detailed in Chapter 6, are that the image models perform strongly on the benchmark data (98.60 % classification accuracy, with uncertainty scores that detect errors with an AUROC of 0.97), and that fine-tuning changed *how* the open model answers — closer to the textbook, without fabricated source attributions — but not *what* it knows: its clinical correctness did not improve, whereas retrieved textbook passages made GPT-4o more faithful and more correct. Knowledge and provenance therefore have to come from retrieval, and safety from explicit rules.

1.3 Structure of the thesis

Chapter 2 reviews how domain-expert LLMs are built and the related work on blood-smear analysis, uncertainty and interpretability. Chapter 3 describes the data and knowledge sources, and Chapter 4 the architecture of the lab assistant. Chapter 5 gives the training procedures and

the evaluation protocol, and Chapter 6 presents and discusses the results, including the case study and the limitations. Chapter 7 concludes and outlines future work.

2 Theoretical Background

2.1 Large language models and their limits in medicine

A large language model is a transformer network trained to predict the next token of text on a very large corpus and then further trained to follow instructions and hold a dialogue. Models such as GPT-4o [33] and the open-weight Llama 3 family [6] can answer questions, summarise documents and produce structured output such as JSON. Singhal *et al.* showed that instruction-tuned LLMs encode a substantial amount of clinical knowledge and can answer medical exam questions at a high level [1].

The same work, and a large literature on hallucination [2], also shows the limits. An LLM generates the most plausible continuation of a text, not the most correct one. When its internal knowledge is incomplete it may produce a confident but false statement, and when asked for references it may produce citations that look real but do not exist [3]. The model also has no access to information outside its prompt, so it cannot know the data of a specific patient unless that data is given to it. For a clinical assistant these are serious problems: a fluent wrong answer is more dangerous than no answer, because it is more likely to be believed.

2.2 Building domain experts from general LLMs

There are three main ways to make a general LLM behave as an expert in a narrow field. They are complementary, and this thesis uses all three (Table 1).

Prompting. The cheapest approach is a carefully written system prompt ("You are a clinical hematology expert…"), together with rules and an output format. Prompting changes the style and focus of the answers, but it does not add knowledge the model lacks, it does not provide sources, and it does not prevent unsupported statements.

Retrieval-augmented generation (RAG). RAG [7] leaves the model unchanged and supplies relevant passages from a curated corpus at question time. The source documents are split into chunks, each chunk is converted into a vector by a sentence-embedding model [8], and the vectors are stored in a vector database. For each question, the chunks whose vectors are most similar to the question's vector are retrieved and placed in the prompt, together with an instruction to answer from them and to cite them. RAG has two important properties for a domain expert: the knowledge can be updated without retraining, and every answer can be checked against the passages it was based on. Comparative studies have found retrieval to be more reliable than fine-tuning for injecting new factual knowledge into an LLM [9].

Domain fine-tuning. Fine-tuning changes the model's weights using examples from the domain. Meditron [10], for example, continued the pre-training of Llama 2 on medical text, while Med-PaLM [1] kept its base model frozen and learned only a small set of prompt parameters (instruction prompt tuning). Full fine-tuning of a model with billions of parameters is too expensive for most projects. *Parameter-efficient* methods solve this. LoRA [11] freezes the original weight matrices W and learns a low-rank update, so that a layer computes

$$h = Wx + \frac{\alpha}{r} BAx, \qquad B \in \mathbb{R}^{d \times r},\ A \in \mathbb{R}^{r \times k},\ r \ll \min(d,k), \tag{1}$$

where only A and B are trained. QLoRA [12] additionally stores the frozen model in 4-bit precision, which makes it possible to fine-tune an 8-billion-parameter model on a single 16 GB GPU such as the free NVIDIA T4 in Google Colab. Fine-tuning is particularly effective at teaching *behaviour*: answer length and structure, terminology, citation format and cautious phrasing. It also produces an open model that can be run on local hardware, independent of a commercial API.

Synthetic training data. When no suitable human-written training data exist, question–answer pairs can be generated by a stronger model from trusted source text, in the spirit of Self-Instruct [13]. This is fast and cheap, but the generated data inherit the generator's mistakes and style, which must be considered when the resulting model is evaluated.

Table 1: Ways of turning a general LLM into a domain expert, and how this thesis uses them.

	Adds knowledge?	**Gives checkable sources?**	**Role in this thesis**
Prompting	No	No	System prompts and JSON output schema
RAG	Yes, at query time	Yes, retrieved passages	Textbook knowledge base (Sec. 4.4)
Fine-tuning (QLoRA)	Mostly style and behaviour	No (model may imitate a citation format)	Open, self-hostable expert (Sec. 4.6)
Tools / agent	Yes, case data and reference values	Yes, logged tool calls	GPT-4o ReAct agent (Sec. 4.5)

2.3 Tool use and the ReAct agent paradigm

Standard RAG performs one retrieval followed by one generation. ReAct [14] generalises this by interleaving reasoning and acting: the model writes a thought, chooses an action (a call to a tool), observes the result, and repeats until it produces a final answer. Tools can be database

searches, calculators or reference tables. For a domain expert this has two advantages. The model can look up exact reference values and case data instead of relying on memory, and every tool call is recorded, so the path to the final answer can be audited. LangGraph [34] provides an implementation of this loop with structured tool calling, which is used in this thesis.

2.4 Evaluating a domain expert

A domain expert can be evaluated in three complementary ways:

1. **Multiple-choice questions** with known answers, such as the medical entrance-exam questions of MedMCQA [15]. They give an objective accuracy, but test recognition of facts rather than explanation.

2. **Similarity of free-text answers to reference answers**, measured by word overlap (ROUGE-L, based on the longest common subsequence [16]) or by the cosine similarity of sentence embeddings. These measures are cheap and reproducible, but they reward answers that *look like* the reference, not necessarily answers that are correct.

3. **Graded judgements** of answers by clinicians or, more cheaply, by a strong LLM used as a judge [17]. An LLM judge scales well but has its own biases and does not replace expert review.

This thesis uses all three: MedMCQA questions for the first, ROUGE-L and embedding similarity for the second, and GPT-4o-mini as an LLM judge for the third (no clinicians rated the answers). In addition, answers are inspected qualitatively (Section 5.4).

2.5 Automated peripheral blood-smear analysis

In a peripheral blood smear, a drop of blood is spread on a glass slide, stained with a Romanowsky-type stain and examined under a microscope. A trained reviewer estimates red-cell morphology and platelet numbers and counts the proportions of the five main white-cell types — neutrophils, lymphocytes, monocytes, eosinophils and basophils — and looks for immature granulocytes, blasts or atypical lymphocytes, which can be the first sign of infection, leukaemia or bone-marrow disease. Digital-morphology analysers are in routine laboratory use, and research on automated smear analysis has moved from hand-crafted image features to deep learning. Acevedo *et al.* showed that convolutional neural networks can classify single blood-cell images with very high accuracy [4] and published the PBC dataset of 17,092 cell images in eight classes, recorded with a CellaVision DM96 analyser [18]. For locating cells in a whole field of view, general object detectors of the YOLO family [35] are fine-tuned on annotated smears such as BCCD [36] and the more recent TXL-PBC collection [19].

Most of this work stops at a label per cell; the interpretation of the differential as a whole is left to the human reader. Table 2 places the present system among typical approaches. The comparison is qualitative: it describes capabilities of system families, not of every individual system.

Table 2: Qualitative comparison of system families.

System family	Image analysis	Uncertainty	Explanation	Clinical interpretation
General LLM chat (e.g. ChatGPT)	None (text only)	None	Free text, no sources	Generic, not case-specific
CNN cell classifiers	Pre-cropped cells	Usually softmax only	Rare or post-hoc	None
Detector + classifier pipelines	Detection and classification	Rarely passed on	Boxes, sometimes saliency	None
This thesis	YOLOv8s + EfficientNet-B0	MC-Dropout level per cell, review flag	Boxes, Grad-CAM, citations, tool trace	RAG + tool-using LLM expert; QLoRA open model

2.6 Uncertainty and explainability

A neural network's softmax output is not a calibrated probability; modern networks tend to be over-confident [20]. Bayesian deep learning treats the network weights as random variables, but exact inference is intractable. Practical approximations include deep ensembles [21], which train several networks and average them, and *Monte Carlo Dropout* [22], which keeps dropout active at prediction time and averages T stochastic forward passes. MC-Dropout needs no retraining when the network already contains dropout layers, which is why it is used here; its disadvantage is that it is only an approximation and is usually less well calibrated than ensembles.

Grad-CAM [23] produces a heatmap showing which image regions most influenced a class prediction. It weights the feature maps A^k of a convolutional layer by the average gradient of the class score y^c:

$$L^c_{\text{Grad-CAM}} = \text{ReLU}\Big(\sum_k \alpha^c_k A^k\Big), \qquad \alpha^c_k = \frac{1}{Z}\sum_{i,j}\frac{\partial y^c}{\partial A^k_{ij}}. \tag{2}$$

In a clinical setting Grad-CAM is most useful as a check: if the heatmap highlights the background or a neighbouring cell rather than the cell of interest, the prediction should be treated as suspect.

2.7 Trustworthy AI in medical decision support

Regulatory and policy documents — the FDA's Good Machine Learning Practice principles [29], the EU Artificial Intelligence Act [30] and the WHO guidance on large multi-modal models in health [31] — converge on a few requirements for high-risk medical AI: transparency about inputs and limits, auditability of decisions, communication of uncertainty, and effective human oversight. These requirements guided the design of the system in this thesis. The system is a research prototype: it is not clinically validated, not a medical device, and not intended for use on patient data outside a research setting.

3 Data and Knowledge Sources

The system uses five data sources: two for the domain expert's knowledge, one for evaluating it externally, and two for training the vision models that provide case evidence (Table 3).

Table 3: Data sources used in the thesis.

Source	Content	Used for
Hematology textbooks [24, 25]	541 PDF pages, 1,049 text chunks	RAG knowledge base; source of Q&A pairs
Synthetic hematology Q&A	2,098 question–answer pairs generated from the chunks	Fine-tuning (1,898) and held-out test (200)
MedMCQA [15]	Medical entrance-exam MCQs; 100-question hematology-related subset	External test of the fine-tuned model
TXL-PBC [19]	1,260 smear images, 18,143 boxes (WBC, RBC, platelet)	Training/testing the cell detector
PBC [18]	17,092 single-cell images, 8 classes	Training/testing the WBC classifier

3.1 Hematology textbook corpus

The domain knowledge of the expert comes from two hematology textbooks: *Essentials of Haematology* by Kawthalkar [24] (511 PDF pages) and a 30-page excerpt of the *Concise Guide to Hematology* edited by Schmaier and Lazarus [25]. Both books are copyrighted. They were used locally, for non-commercial research, and are not redistributed with the project's code.

Text was extracted page by page with the `pypdf` library, whitespace was normalised, and the text was split into chunks of 200 words with an overlap of 40 words between consecutive chunks. The chunk size was meant to keep each chunk within the 256-token input limit of the original embedding model, `all-MiniLM-L6-v2`, but a later check showed that this assumption did not hold: the limit counts word-piece tokens, not words, and hematology terms are often split into several pieces. The chunks contain on average 343 tokens (median 334), and 1,035 of the 1,049 chunks (98.7 %) are longer than 256 tokens, so the original embedder ignored about a quarter of the knowledge-base text (25.4 % of all tokens) when building the index; the full chunk was still passed to the language model when it was retrieved (script `scripts/check_chunk_tokens.py`). The embedding model was therefore replaced by one that reads 512 tokens, which only 4 chunks exceed (Section 4.4); the chunks themselves were not changed. Tables and figures were not parsed and no optical character recognition was performed. The result is 1,049 chunks: 996 from the first book and 53 from the second. Each chunk is stored with its source file name and a chunk

index (page numbers are not stored).

3.2 Synthetic question–answer dataset

No public collection of hematology question–answer pairs matched the content of the two textbooks, so a training set was generated from the knowledge base itself (script `scripts/build_finetune_dataset.py`). For every chunk, an OpenAI GPT model[1] received the passage and the instruction to produce two "grounded" question–answer pairs: realistic questions that the passage can answer, with 2–5-sentence answers based *only* on the passage, varying between definitions, mechanisms, differentials, laboratory interpretation and short patient vignettes. Each answer ends with a source line such as "*Source: essentials_haematology.pdf.*" Every training example is stored as a three-message chat (system prompt, question, answer); the final version of the data also stores the chunk each pair was generated from (`source`, `chunk_id`). An example record:

```
{"messages": [
  {"role": "system",
   "content": "You are a clinical hematology expert. Always ground answers
              in the provided textbook reference. Use cautious, ..."},
  {"role": "user",
   "content": "What genetic defect is responsible for 40-50% of severe
              cases of haemophilia A?"},
  {"role": "assistant",
   "content": "An inversion in the F VIII gene at intron 22 is responsible
              for 40-50% of cases of severe haemophilia A.
              \n\nSource: essentials_haematology.pdf."}]}
```

The data were generated twice (see the split below). Both versions contain 2,098 pairs, exactly two for each of the 1,049 chunks; in the final version, questions are short (13.6 words on average) and answers are concise (39.4 words on average).

Train/test split. The pairs are split into 90 % training and 10 % test data by groups of textbook passages, so that no test question is about a passage seen during fine-tuning. The first version of this split relied on an assumption — that records $2i$ and $2i+1$ of the generated file come from the same chunk — which turned out to be false: the generation script collected its results in completion order and then shuffled all records, so that split was in effect random. A check made during the preparation of this thesis (script `scripts/check_split_leakage.py`) estimated that about 90 of the 100 evaluated test questions had a *sibling* question from the same passage in the training data. The data were therefore regenerated with an explicit chunk label, and the split

[1] The script's default generator is `gpt-4o-mini`.

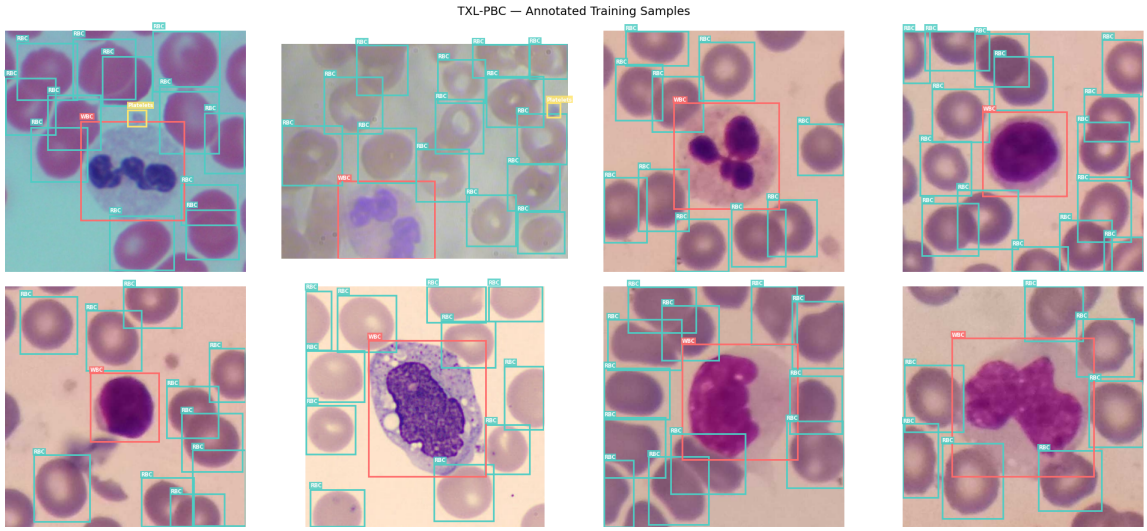
was redone over **blocks of 10 consecutive chunks** of the same book: 10 of the 106 blocks were held out (seed 42). Whole blocks are used because neighbouring chunks share 40 words; with this split no chunk appears on both sides, only 20 of the 100 test chunks share such an overlap with a training chunk, and no test question text also occurs in training. The split contains 1,898 training and 200 test pairs (180 and 20 of the test pairs come from the first and the second book, respectively; `results/domain_expert_v2/split_check.json`). All domain-expert results in Chapter 6 come from this corrected split; Section 6.3 compares them with the first, leaky run.

3.3 External multiple-choice questions (MedMCQA)

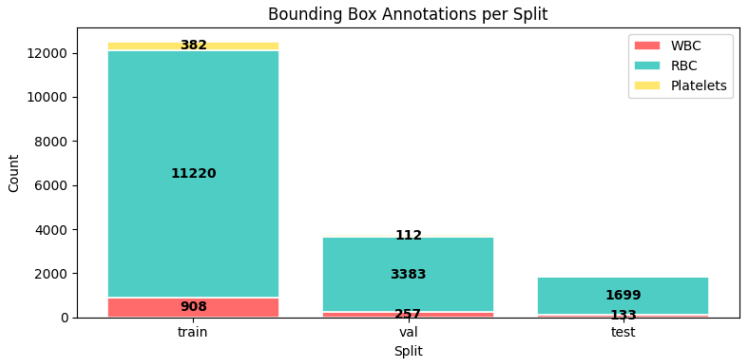
To test whether fine-tuning changes general hematology knowledge — and not only the ability to reproduce the textbook's answers — the fine-tuned model was also evaluated on questions that were not derived from the two books. MedMCQA [15] contains multiple-choice questions from Indian medical entrance examinations, each with four options and one correct answer. From its validation split, single-answer questions whose topic or question text matched hematology keywords (e.g. *haemat-*, *anaemi-*, *leuk-*, *platelet*, *coagul-*, *thalass-*) were selected, and 100 were used. The keyword filter is not perfect: manual inspection shows that roughly a quarter of the selected questions are only loosely related to hematology (for example, questions matched through *hematuria* or *myelopathy*). The subset is therefore described as "hematology-related" in the results.

3.4 Cell detection data: TXL-PBC

TXL-PBC [19] integrates and re-annotates images from four public blood-cell resources (BCCD, BCDD, PBC and Raabin-WBC). It contains 1,260 smear images with 18,143 bounding boxes in three classes — white blood cells (WBC), red blood cells (RBC) and platelets — and is used with its published split of 882 training, 252 validation and 126 test images. The class distribution is dominated by red cells, as in real smears (Figure 1). Annotations are in YOLO format (one text file per image with normalised box coordinates).



(a) Annotated training images.



(b) Box annotations per split.

Figure 1: The TXL-PBC detection dataset: (a) example training images with their WBC, RBC and platelet boxes; (b) number of annotated boxes per class and split, counted from the raw label files (before Ultralytics removed a small number of duplicate labels). Red cells dominate every split.

3.5 White-cell classification data: PBC

The PBC dataset of Acevedo *et al.* [18] contains 17,092 single-cell images of 360×363 pixels, recorded with a CellaVision DM96 analyser at the Hospital Clínic of Barcelona from individuals without infection, hematological disease or pharmacological treatment. Each image was labelled by clinical pathologists into one of eight classes: *basophil, eosinophil, erythroblast, immature granulocyte (IG), lymphocyte, monocyte, neutrophil* and *platelet*. The classes are moderately imbalanced (about 3,300 neutrophils and 1,200 basophils). The images were resized to 224×224 pixels. The final classifier (v2, Section 5) uses a class-stratified split into 11,963 training, 1,710 validation and 3,419 test images (70/10/20, seed 42); an earlier run (v1) used an unstratified 80/20 train/test split. Both splits are at image level, not grouped by donor, because the dataset does not identify the donor of each image.

4 System Architecture and Design

4.1 Architectural overview

The lab assistant consists of two parts (Figure 2). Two *image-analysis stages* turn a smear image into structured findings: a detector that finds and counts the cells, and a classifier that assigns a type, an uncertainty level and a saliency map to every white cell. The *domain expert* then interprets these findings using hematology knowledge. The expert can be run with a hosted model (GPT-4o inside a tool-using agent) or with the open, fine-tuned Llama model, and it can also be used on its own as a hematology question-answering system.

The pipeline is strictly feed-forward: every stage has a typed input and output, and each stage consumes only the structured output of the previous one, never the raw upstream data. This keeps each stage replaceable — for example, the detector can be exchanged without retraining the classifier, and the language model can be exchanged without touching the vision stages — and it produces an inspectable artefact at every step: a box overlay, a label with an uncertainty level, a heatmap, retrieved passages and a tool trace.

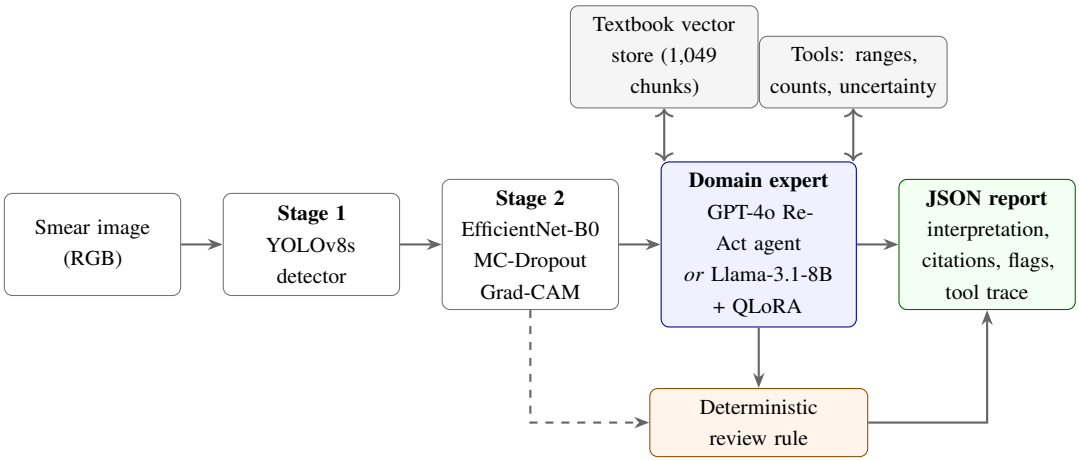


Figure 2: Architecture of the system. Stage 1 returns boxes, classes and confidences; Stage 2 returns, for every white cell, a label, confidence, entropy, uncertainty level and heatmap. The expert queries the knowledge base and tools and writes the report. The dashed arrow shows that the review rule reads the Stage-2 results directly, independently of the language model. The Llama backend was evaluated stand-alone, not inside the running system (Section 7).

The components of the thesis title correspond to the parts of the system as shown in Table 4. The word *multimodal* is used at the level of the system: the image and the text are processed by different specialised models and fused in the reasoning stage, rather than by a single vision-language model.

Table 4: How the parts of the thesis title map onto the system.

Title component	Realisation in this thesis	Where
Hybrid	Combines learned models (CNNs, LLMs) with deterministic rules and tools; uses a hosted model (GPT-4o), and a self-hostable fine-tuned open model (Llama-3.1-8B) was built and evaluated stand-alone	Ch. 4
Multimodal	Fuses image evidence (the smear) with textual knowledge (textbooks) and, optionally, tabular laboratory values (CBC)	Sec. 4.4, 4.8
Lab assistant	End-to-end pipeline with a REST backend and web interface that analyses an uploaded smear	Sec. 4.8
Blood image classification	YOLOv8s cell detection and EfficientNet-B0 white-cell classification with uncertainty and saliency	Sec. 4.2, 4.3
Domain-expert LLM reasoning	GPT-4o ReAct agent with hematology tools; QLoRA-fine-tuned Llama-3.1-8B	Sec. 4.5, 4.6
RAG	Vector store of 1,049 textbook passages, retrieved and cited at answer time	Sec. 4.4

All runtime parameters — model paths, thresholds, the number of MC-Dropout passes, chunk size, embedding model, number of retrieved chunks, language model, temperature and agent budget — are set in a single `config.yaml` file. All inference goes through one orchestrator class, `BloodSmearPipeline`; the command-line interface and the web backend are thin wrappers around it.

4.2 Stage 1: cell detection with YOLOv8

Given an image I, Stage 1 returns a set of boxes $\mathscr{B}(I) = \{(x_1, y_1, x_2, y_2, c, p)_i\}$, where $c \in \{\text{WBC}, \text{RBC}, \text{Platelet}\}$ and p is the detector's confidence. Boxes with $p < 0.50$ are discarded.

The detector is YOLOv8s [35], the "small" variant (about 11 million parameters) of a single-stage, anchor-free object detector. It consists of a convolutional backbone, a feature-pyramid neck that fuses information from several scales, and a decoupled head that predicts class scores and box coordinates. The small variant was chosen as a compromise between accuracy and speed on an ordinary laptop CPU. At inference the checkpoint is loaded once and kept in memory, and class names are normalised (*Platelets* and *Platelet* are unified, because source datasets use both). The white-cell boxes are cropped from the image and passed to Stage 2, and the per-class counts are passed to the domain expert.

4.3 Stage 2: white-cell classification with uncertainty and saliency

For each WBC crop, Stage 2 returns

$$y(\text{crop}) = \big(\hat{c},\ \bar{p}_{(1)},\ H,\ v,\ b,\ \text{Grad-CAM}\big), \tag{3}$$

the predicted class, its mean probability (confidence), the predictive entropy, the mean variance across passes, an uncertainty level $b \in \{\text{LOW}, \text{MEDIUM}, \text{HIGH}\}$, and a saliency overlay.

Classifier. The backbone is EfficientNet-B0 [26] from the `timm` library, pre-trained on ImageNet. Its 1,000-class head is replaced by a small multilayer head: Linear(1280→1024) – ReLU – Dropout(0.3) – Linear(1024→512) – ReLU – Dropout(0.3) – Linear(512→8). The two dropout layers are what makes MC-Dropout possible.

Monte Carlo Dropout. At prediction time the network is put into evaluation mode and then only its `nn.Dropout` modules are switched back to training mode; batch-normalisation layers stay in evaluation mode so that their statistics do not drift. Each crop is passed through the network $T = 20$ times, giving a $T \times C$ matrix of class probabilities. The mean $\bar{p}$ over the passes is the prediction, $\bar{p}_{(1)}$ is its largest entry, $H = -\sum_c \bar{p}_c \log \bar{p}_c$ is the predictive entropy, and v is the per-class variance across passes averaged over the classes. The uncertainty level is then

$$b = \begin{cases} \text{LOW} & \text{if } \bar{p}_{(1)} \geq 0.85 \text{ and } H < 0.3, \\ \text{MEDIUM} & \text{else if } \bar{p}_{(1)} \geq 0.65 \text{ and } H < 0.6, \\ \text{HIGH} & \text{otherwise.} \end{cases} \tag{4}$$

A HIGH cell is marked as *flagged*. The thresholds are set in the configuration file; they were chosen by hand, and their behaviour on held-out data is evaluated in Section 6 (Table 10).

Grad-CAM. For each crop, the predicted class score is back-propagated to the network's final convolutional layer (`conv_head`), the Grad-CAM map (Equation 2) is up-sampled to 224×224 and blended over the crop with an opacity of 0.45. The resulting image is returned with the prediction and shown in the user interface.

4.4 Stage 3: domain-expert reasoning — knowledge base and retrieval (RAG)

The 1,049 textbook chunks (Section 3.1) are embedded with `bge-small-en-v1.5` [28], a small BERT-type retrieval model that maps a text to a 384-dimensional vector, reads up to 512 tokens

and runs quickly on a CPU. Following the model's documentation, each *query* (not the chunks) is prefixed with the instruction "Represent this sentence for searching relevant passages:". This model replaced the sentence-transformer `all-MiniLM-L6-v2` [8, 27] of the first version, which reads only 256 tokens and therefore ignored the end of almost every chunk (Section 3.1); the comparison that led to the change is reported in Section 6.4. The vectors are stored in a persistent ChromaDB collection [37] with cosine distance, so the index is built once and re-used; it is rebuilt automatically if the number of stored chunks does not match the corpus or if it was built with a different embedding model. For a query, the query text is embedded with the same model and the five most similar chunks are returned with their source and chunk index. The complete pipeline is shown in Figure 3. An optional web-search extension, restricted to an allow-list of medical domains (e.g. NCBI, MedlinePlus, WHO), exists but is disabled in all experiments for reproducibility.

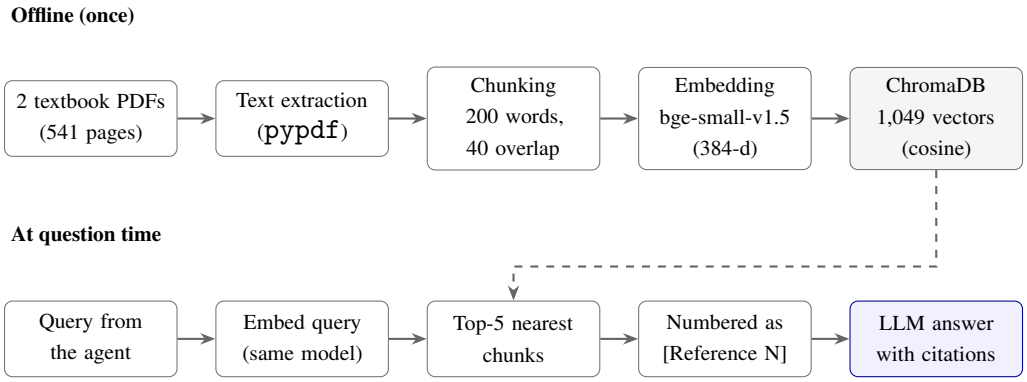


Figure 3: The retrieval-augmented generation (RAG) pipeline. The knowledge base is built once; at question time the agent's query is embedded with the same model and the five most similar chunks are returned, numbered so that the model can cite them.

4.5 Stage 3, backend A: GPT-4o with tools (ReAct agent)

The main runtime expert is GPT-4o [33], called at temperature 0.1 through the OpenAI API. GPT-4o was chosen because it follows structured-output and tool-calling instructions reliably and needs no local GPU. The model is wrapped in a LangGraph ReAct agent [14, 34] with the five tools listed in Table 5.

Table 5: Tools available to the domain-expert agent.

Tool	Purpose
`query_knowledge_base`	Takes a search query; retrieves the top-5 textbook chunks and numbers them as `[Reference N]`.
`lookup_lab_reference_ranges`	Takes a parameter name; returns a typical adult reference range (e.g. monocytes 2–8 % of WBC).
`interpret_differential`	Takes the differential as JSON; compares each class percentage with its reference range and reports LOW/HIGH/normal.
`get_uncertainty_summary`	Returns the Stage-2 MC-Dropout statistics (number of flagged cells).
`get_detection_counts`	Returns the Stage-1 cell counts for the field of view.

The system prompt tells the agent to call the knowledge base one to three times to ground its claims, to look up reference ranges before calling a value abnormal, to review the uncertainty summary before making confident statements, to cite evidence as `[Reference N]`, to give differential rather than definitive diagnoses, and to remember that the counts are per microscope field and must not be converted into clinical concentrations. The final message must be a JSON object with seven fields: the interpretation, key findings, differential diagnoses, recommendations, safety flags, the review flag `requires_expert_review` and a confidence assessment. The number of reasoning steps is bounded through LangGraph's recursion limit (derived from a configured maximum of six iterations), and every tool call and tool result is returned to the user as the *agent trace*. The knowledge-base tool shows the agent only the first 600 characters of each retrieved chunk. Because a model can write "[Reference N]" without having retrieved anything, a deterministic check, added after the case study (Section 6.6), compares every cited number with the passages the agent actually retrieved; an unsupported citation adds an `UNSUPPORTED_CITATION` flag and forces expert review.

A simpler *linear* mode is also available: one retrieval followed by one LLM call with the retrieved chunks in the prompt. In linear mode the code checks that the cited reference numbers actually exist among the retrieved chunks, and it adds an `INSUFFICIENT_EVIDENCE` flag if no valid citation or no differential diagnosis is returned.

The language-model client talks to any OpenAI-compatible endpoint. Replacing GPT-4o by a self-hosted model therefore only requires changing the model name in `config.yaml` and the `OPENAI_BASE_URL` environment variable.

4.6 Stage 3, backend B: a fine-tuned open-weight model

The second form of the expert is an open-weight model adapted to the domain, which can run on local hardware. The base model is **Llama-3.1-8B-Instruct** [6], loaded in 4-bit precision, and it is adapted with **QLoRA** [12]: LoRA adapters of rank $r = 16$ ($\alpha = 32$, no dropout) are attached to all attention and feed-forward projections (`q, k, v, o, gate, up, down`), which adds 41.9 million trainable parameters — 0.52 % of the model's 8.07 billion. The adapter is trained on the 1,898 training pairs (Section 3.2), so the model learns to answer hematology questions in the form of the textbook-grounded answers. The resulting adapter (about 42 million parameters, roughly 84 MB in 16-bit precision) is published at `https://huggingface.co/Afridi07/hematology-llama-3.1-8b-lora` and could be served, for example with vLLM, behind an OpenAI-compatible endpoint and selected in the configuration. This was not tested: the model was evaluated stand-alone, and it was not trained to call tools or to produce the agent's JSON format. Training details are given in Section 5.3.

It is important to note what the training examples contain: a system prompt, a question and an answer — but *not* the textbook passage. The fine-tuned model therefore learns to answer from its weights, not to read and cite a retrieved passage. This design choice turns out to matter for how its citations should be interpreted (Section 6.3).

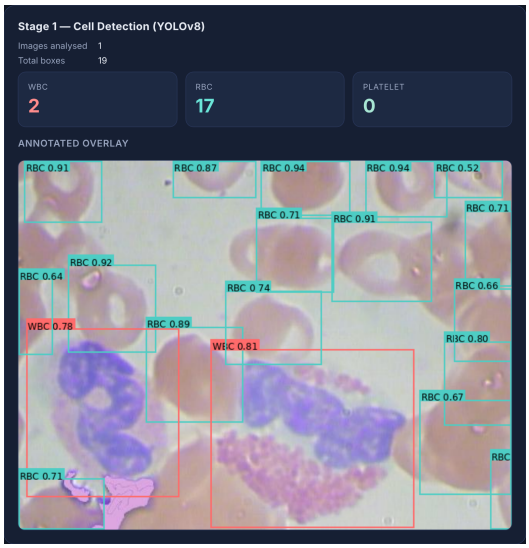
4.7 Safety logic: the expert-review rule

Two design principles guide the safety behaviour. First, safety-relevant information is always passed forward and shown: every detection carries its confidence, every classified cell its uncertainty level and heatmap, every interpretation its tool trace. Second, the decision whether a human must review the case is not left to the language model alone. The flag `requires_expert_review` starts from the model's own judgement, but deterministic code *forces it to true* whenever at least one Stage-2 cell is flagged as HIGH uncertainty, and adds a `HIGH_UNCERTAINTY` safety flag. In linear mode, missing citations also force the flag, and in agent mode so does a citation of a passage the agent never retrieved (Section 4.5). A second rule, added after the case study in Section 6.6 revealed the gap, checks the *number* of classified white cells: if it is below a configurable minimum (`llm.safety.min_wbc_for_differential`, default 100, following the convention of counting at least 100 cells for a differential [32]), an `INSUFFICIENT_CELL_COUNT` flag is added and review is forced. Both reasoners are also told the cell count and instructed not to interpret percentages when it is insufficient. The uncertainty rule is applied in the post-processing of each reasoner and the minimum-cell rule by the pipeline after either reasoner has answered (`src/utils/safety_rules.py`), so both hold in both modes. The code can only raise the flag, never lower it. Stage-1 detection confidence does not influence the flag.

4.8 Software implementation and user interface

The system is implemented as a Python library (`src/`) with a FastAPI backend (`backend/`) and a React + TypeScript web interface (`frontend/`). The backend exposes `GET /api/health`, `GET /api/samples` and `POST /api/analyze`; the latter accepts an image (and optionally a JSON object with CBC laboratory values) and returns the full JSON result. The pipeline is loaded lazily on the first request. The interface shows five panels: the detection overlay with cell counts, the WBC differential with per-cell uncertainty, the Grad-CAM heatmaps, the expert's interpretation with citations and safety flags, and the agent trace (Figures 4 and 5). Forty automated `pytest` tests cover configuration loading, batch logic, parsing of model output, retrieval fall-backs, the rebuilding of the vector index when the embedding model changes, the uncertainty schema, the CBC analyser, the minimum-cell rule, the agent's citation check and the prompts given to the reasoners. All 40 tests pass; they test the logic with small stand-ins and do not load the trained checkpoints, which were exercised by complete pipeline runs (Section 6.6). The training notebooks for all three models are included and can be run on Google Colab.

The optional CBC input deserves a remark. A small module converts laboratory values (WBC, hemoglobin, MCV, platelets, etc.) into findings such as *leukocytosis* or *microcytosis* with a mild/moderate/severe grade. The findings are added to the prompt of both reasoners (in the agent's input message, and in linear mode also to the retrieval query), together with an instruction to reconcile them with the image findings. The web interface does not yet have a CBC input field, so CBC values can currently only be supplied through the API. Because no experiment in this thesis uses CBC values, this feature is presented as an implemented extension, not as an evaluated result.

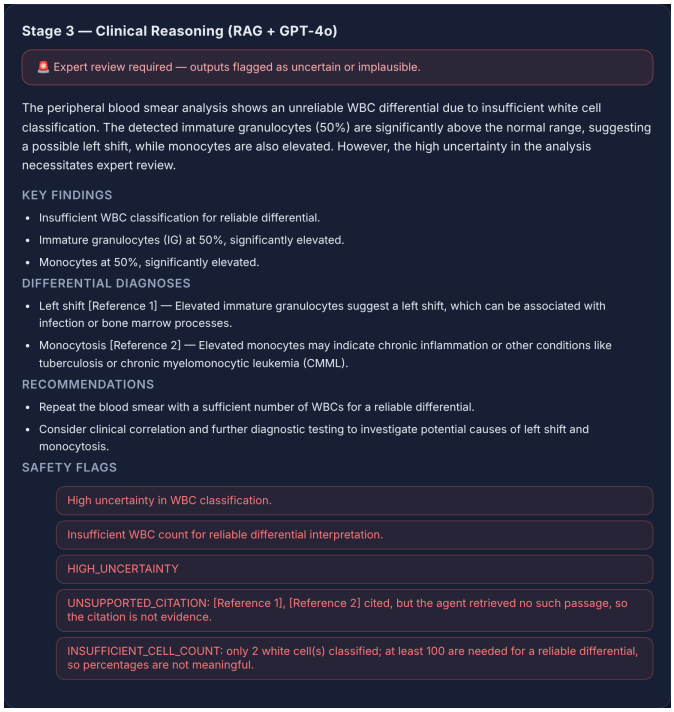


(a) Detection panel: counts and box overlay.



(b) Differential, uncertainty and Grad-CAM; the first cell is flagged HIGH.

Figure 4: Web interface, image-analysis stages: the final system on the case-study image BCCD_00010 (Section 6.6), rendered from the saved run in `results/case_study_rerun/`.



(a) Interpretation with the forced review and the safety flags.



(b) Agent trace: reference ranges and uncertainty were consulted, the knowledge base was not.

Figure 5: Web interface, domain-expert output for the same run. The `UNSUPPORTED_CITATION` flag marks citations of passages the agent never retrieved.

5 Research Methodology

This chapter describes how the three models were trained and how the image-analysis stages and the domain expert were evaluated. All training was done on Google Colab with a single NVIDIA T4 GPU (16 GB).

5.1 Training the image-analysis models

YOLOv8s. The detector was fine-tuned from the COCO-pre-trained `yolov8s.pt` checkpoint for 50 epochs (0.30 hours on the T4) with the settings in Table 6. All flips and rotations are physically plausible for a smear, which has no preferred orientation.

Table 6: YOLOv8s training settings (from the Ultralytics training log).

Setting	Value	Setting	Value
Epochs	50	Image size	640×640
Batch size	8	Optimiser	AdamW
Initial LR	0.001	Final LR factor	0.01 (linear)
Momentum	0.937	Weight decay	5×10^{-4}
Warm-up epochs	3	Mosaic	1.0 (off in last 10 epochs)
HSV jitter (h/s/v)	0.015 / 0.7 / 0.4	Rotation	$\pm 10^{\circ}$
Flip (horizontal/vertical)	0.5 / 0.5	Seed	0 (deterministic)

EfficientNet-B0. The classifier was trained in two phases (the same recipe was used in both training runs described below). In Phase 1 the backbone was frozen and only the new head was trained (40 epochs at most, batch size 32, Adam with learning rate 10^{-3}, learning rate halved when accuracy stopped improving for three epochs, early stopping with patience five). In Phase 2 all layers were unfrozen and trained for 10 epochs with a ten times smaller learning rate (10^{-4}). The augmentations were random horizontal (p = 0.5) and vertical (p = 0.3) flips, random affine transforms ($\pm 25^{\circ}$, scale 0.8–1.2) and colour jitter.

Two training runs. The *original run (v1)* used an 80/20 train/test split and had no separate validation set: the accuracy monitored after every epoch, the learning-rate scheduler and the choice of the saved checkpoint were all based on the *test* images, so its test accuracy is optimistic. This flaw was found during the preparation of the thesis, and the classifier was therefore retrained in a *corrected run (v2)* with the same architecture and recipe but a proper protocol: a class-stratified 70/10/20 split into 11,963 training, 1,710 validation and 3,419 test images (sorted file lists, seed 42), the learning-rate schedule, early stopping and checkpoint choice based on the validation set only, and a single evaluation on the test set. One non-image file (a macOS `.DS_` metadata file with a `.jpg` extension) was skipped. All classification results reported in Chapter 6 are from v2 unless stated otherwise; the v2 checkpoint is also the one configured in the system.

5.2 Evaluation of the image-analysis stages

Detection. The YOLOv8s model was evaluated once on the 126-image TXL-PBC test split with the standard object-detection metrics: precision, recall, mean average precision at an intersection-over-union threshold of 0.50 (mAP@0.50), and mAP averaged over thresholds 0.50–0.95. The checkpoint was selected on the validation split.

Classification. The EfficientNet-B0 model (v2) was evaluated once on its 3,419 test images with top-1 accuracy, per-class precision, recall and F1, and a normalised confusion matrix.

Uncertainty. The test images were also passed through the classifier with MC-Dropout ($T = 20$, exactly as in the running system) to measure three things: (i) *calibration*, as the expected calibration error (ECE, 15 equal-width confidence bins) and a reliability diagram, for both the MC-Dropout mean and a single deterministic pass; (ii) how well the uncertainty scores *detect errors*, as the area under the ROC curve (AUROC) of $1 - \bar{p}_{(1)}$, entropy and variance for separating misclassified from correctly classified cells; and (iii) the accuracy of the cells in each uncertainty level (LOW, MEDIUM, HIGH) with the thresholds of the configuration file.

5.3 Training the domain-expert model

The Llama model was fine-tuned with the Unsloth library [38] (version 2026.11.2, with Transformers 5.17.0 and PyTorch 2.11) and the supervised fine-tuning trainer of the TRL library, using the settings in Table 7. Each training example was rendered with the Llama-3.1 chat template and the loss was computed on the full sequence. Training took 476 optimisation steps and 49.0 minutes (Colab output saved in `results/domain_expert_v2/colab_run_executed.ipynb`). The loss fell from 2.69 at the first logged step to about 0.56 at the end of the first epoch and 0.41 at the last logged step (Figure 6); the drop at the start of the second epoch is the typical sign that

the model has begun to memorise the training answers, and one reason why more epochs were not used. Figure 7 summarises the whole domain-expert experiment, from data generation and the block-level split to training and the evaluations described next.

Table 7: QLoRA fine-tuning settings.

Setting	Value
Base model	`unsloth/Meta-Llama-3.1-8B-Instruct-bnb-4bit` (4-bit)
LoRA rank / alpha / dropout	16 / 32 / 0.0
Target modules	`q, k, v, o, gate, up, down` projections
Trainable parameters	41,943,040 (0.52 % of 8.07 B)
Training examples	1,898
Epochs / steps	2 / 476
Batch size	2 per device $\times$ 4 gradient accumulation = 8
Learning rate	2×10^{-4}, linear decay, 10 warm-up steps
Optimiser / weight decay	AdamW (8-bit) / 0.01
Maximum sequence length	2,048 tokens
Precision	FP16 (the T4 does not support BF16)
Seed	42
Training time	49.0 min on one T4

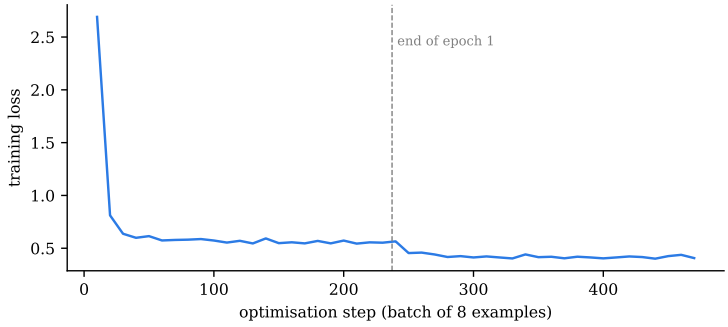


Figure 6: Training loss of the QLoRA fine-tuning, logged every 10 steps.

5.4 Evaluation protocol for the domain expert

The evaluation compares the original **Llama-3.1-8B-Instruct (base)**, the same model with the **QLoRA adapter (ours)**, **GPT-4o** as a reference for a strong hosted model, and **GPT-4o + RAG**, the same hosted model given retrieved textbook passages. The base and fine-tuned answers come from the *same* loaded model with the adapter switched off and on, so any difference is caused by the adapter alone. All systems received the same system prompt

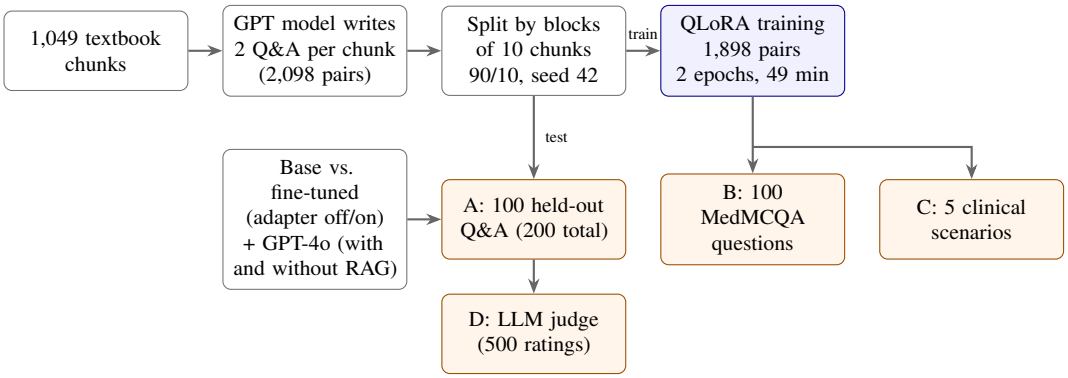


Figure 7: Workflow of the domain-expert experiment: data generation, block-level split, QLoRA training and the evaluations A–D.

as used in training; the first three were asked *without* retrieved passages (this measures the model itself), and decoding was deterministic (greedy decoding for Llama; temperature 0 for GPT-4o). Answers were limited to 256 new tokens for Llama and 300 for GPT-4o. (In the final run the GPT-4o step in Colab failed on an expired API key, so its answers were added afterwards with identical settings by `scripts/add_gpt4o_baseline.py`.) For *GPT-4o + RAG* (`scripts/run_rag_baseline.py`), the system's own retriever (Section 4.4) returned the top five chunks for each question, placed before the question as in the linear mode, with all other settings unchanged; a second variant cut every chunk to 600 characters, as the agent's tool does.

A. In-domain held-out questions. The first 100 of the 200 held-out questions (a notebook setting that kept the GPU evaluation to about half an hour) were answered by all systems and compared with the reference answers using:

- **ROUGE-L F1** [16]: word-level overlap based on the longest common subsequence (with stemming);
- **semantic similarity**: the cosine similarity between the `all-MiniLM-L6-v2` embeddings of the answer and of the reference answer;
- **source marker rate**: the share of answers that contain a source marker ("Source:", "[Reference", or a textbook file name). This measures the citation *format*, not whether the citation is correct;
- **answer length** in words.

B. External multiple-choice questions. The 100 MedMCQA questions were given to the base and fine-tuned models with the four options and the instruction to answer with a single letter; the first letter A–D in the answer was taken as the choice. Random guessing gives 25 %. GPT-4o was not evaluated on this set.

C. Qualitative comparison. Five clinical questions written for this thesis — including one that describes the two-cell differential of the case study — were answered by the base and the fine-tuned model and inspected by hand. In addition, all in-domain answers were checked for references to sources the models had never been given: an explicit rule flags a quoted title after "textbook" or "book" and reference-list entries such as "Lichtman, M. A. (2018)"; every flagged answer was then inspected individually, and the remaining answers were searched for attributions the rule had missed (`flagged_citations_checked.csv`).

Statistics. Because all systems answer the same questions, differences were tested in a paired way: a 95 % bootstrap confidence interval (5,000 resamples of the 100 questions) for the difference in mean ROUGE-L, and an exact McNemar test for the difference in multiple-choice accuracy. All statistics are computed from the saved answers by `scripts/analyze_domain_expert.py` and `scripts/analyze_judge.py` (outputs in `results/domain_expert_v2/`).

D. LLM-as-judge. Following [17], GPT-4o-mini (temperature 0) rated every in-domain answer of every system — 500 ratings — on two 1–5 scales: *faithfulness* (agreement with the textbook reference answer, no unsupported or contradicting claims) and *clinical correctness* (medical accuracy). The judge saw the question, the reference answer and one model answer at a time, never the name of the system, and was told that a source line earns no credit and that length does not matter. The ratings are produced from the saved answers by a stand-alone script (`scripts/judge_domain_expert.py`), because the notebook's own judging step had failed in the first run (a string-formatting error). Differences between systems are reported with paired bootstrap confidence intervals. An LLM judge is not a substitute for clinicians, and GPT-4o-mini may be biased towards the style of its larger sibling GPT-4o.

E. Retrieval quality. Every question of the final Q&A data records its source chunk, so retrieval can be scored directly (`scripts/eval_retrieval.py`): each embedding model ranks the 1,049 chunks for all 2,098 questions, exactly as in the running system, and hit@1, hit@5 (the system retrieves five chunks), MRR@10 and hit@5 including the neighbouring chunk are reported. "Late" questions, whose answer overlaps more with the chunk after its first 150 words, test the effect of truncation; models are compared with an exact McNemar test on hit@5.

6 Results and Discussion

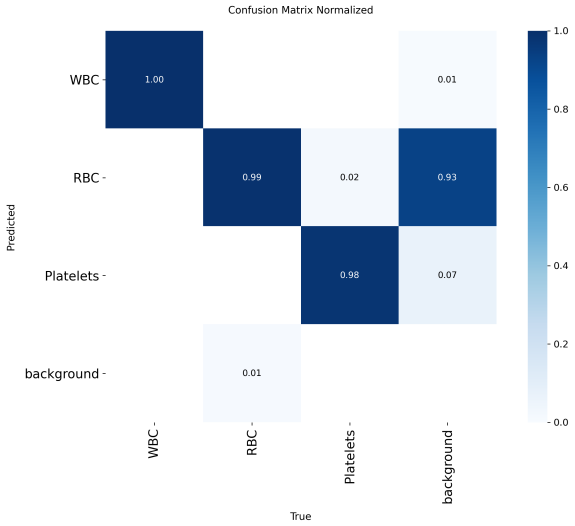
This chapter first presents the results of the two image-analysis stages and then the evaluation of the domain-expert LLM, which is the central question of the thesis topic. It continues with an end-to-end case study of a real run of the lab assistant, and ends with a discussion of the research questions and the limitations.

6.1 Stage 1: detection results

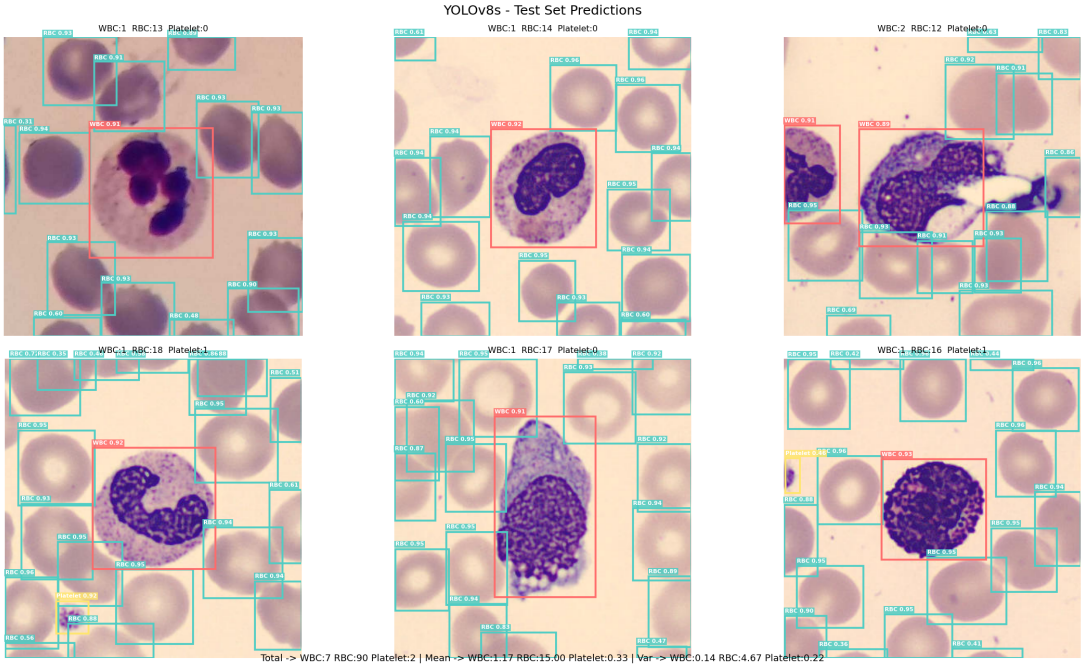
On the 126-image TXL-PBC test split, the detector reached a mAP@0.50 of 0.985 and a mAP@0.50:0.95 of 0.876 (Table 8). White cells were found with a recall of 1.000, so every annotated white cell in the test split reaches the classifier. Red cells have the lowest recall (0.943), as expected for densely packed, overlapping cells, and platelets — the smallest and rarest objects (49 test instances) — have the lowest precision and mAP. The normalised confusion matrix is nearly diagonal; the errors are missed cells and false detections against the background rather than confusions between classes (Figure 8). The training and validation losses decreased steadily over the 50 epochs and the validation metrics levelled off without a late decline, so there is no sign of over-fitting (Figure 9).

Table 8: YOLOv8s results on the TXL-PBC test split.

Class	Images	Instances	Precision	Recall	mAP@0.50	mAP@0.50:0.95
WBC	126	131	0.992	1.000	0.995	0.889
RBC	126	1,688	0.977	0.943	0.990	0.906
Platelet	34	49	0.958	0.939	0.970	0.834
All	126	1,868	**0.976**	**0.961**	**0.985**	**0.876**



(a) Normalised confusion matrix.



(b) Predictions on test images.

Figure 8: Detection results on the TXL-PBC test split.

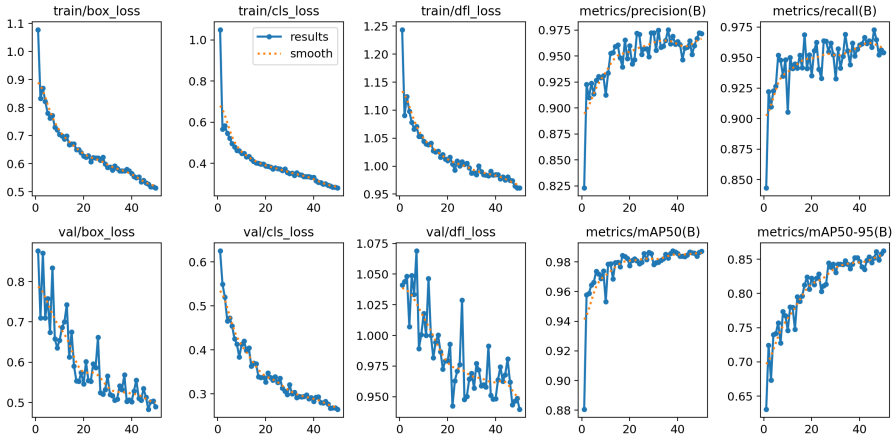


Figure 9: YOLOv8s training curves over 50 epochs: training and validation box, class and distribution-focal losses (left three columns) and validation precision, recall, mAP@0.50 and mAP@0.50:0.95 (right).

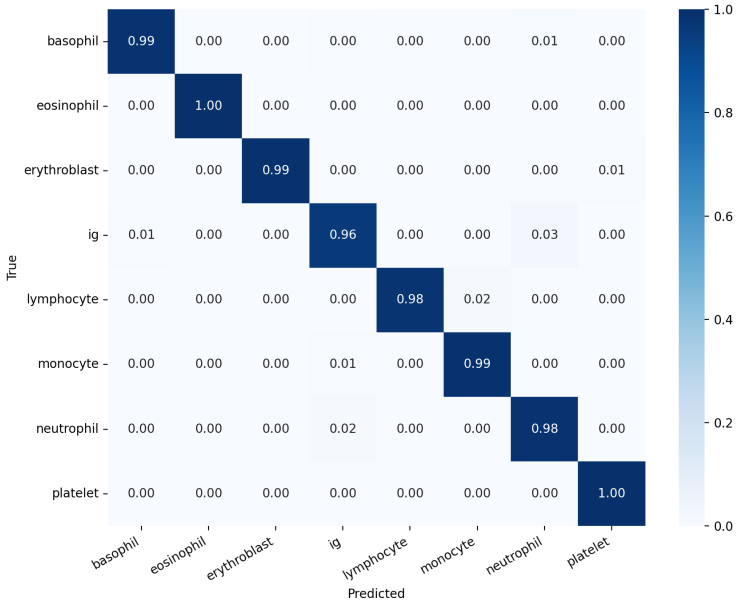
6.2 Stage 2: classification and uncertainty results

With the corrected protocol (v2), EfficientNet-B0 reached a top-1 accuracy of **98.60 %** on the 3,419 held-out test images (98.57 % with MC-Dropout averaging), with a macro-averaged F1 of 0.987 (Table 9). The original run (v1), whose checkpoint had been selected on its own test images, reported 99.44 %; the difference of 0.84 percentage points is the size of the optimism that the flawed protocol introduced, and 98.60 % is the figure this thesis relies on. The best validation accuracy was 87.78 % after Phase 1 (frozen backbone, early stop at epoch 31) and 98.60 % after Phase 2, so unfreezing the backbone again added more than ten percentage points (Figure 10).

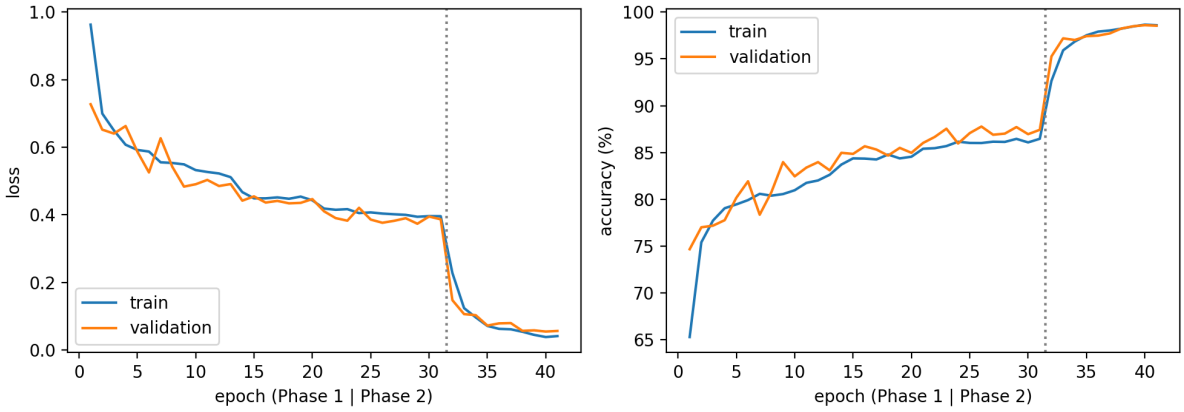
The lowest recall belongs to the immature granulocyte (0.959), and the lowest precision to the neutrophil (0.968): most of the remaining errors are immature granulocytes classified as neutrophils (Figure 10). This is morphologically plausible — immature granulocytes are transitional forms on the way to mature neutrophils, and the boundary is also difficult for human reviewers — and it is exactly the kind of case in which an uncertainty estimate is useful. Figure 11 illustrates the uncertainty levels on eight test images; it was produced with the original (v1) model, before the systematic evaluation of the v2 model that follows.

Table 9: Per-class results of EfficientNet-B0 (v2) on its held-out test set.

Class	Precision	Recall	F1	Support
Basophil	0.984	0.988	0.986	244
Eosinophil	1.000	1.000	1.000	623
Erythroblast	0.997	0.990	0.994	310
Immature granulocyte	0.977	0.959	0.968	579
Lymphocyte	0.996	0.984	0.990	243
Monocyte	0.983	0.989	0.986	284
Neutrophil	0.968	0.985	0.976	666
Platelet	0.996	0.998	0.997	470
Accuracy		**0.9860**		3,419



(a) Normalised confusion matrix (test set).



(b) Training and validation loss and accuracy over Phase 1 and Phase 2 (dotted line).

Figure 10: Classification results of the corrected training run (v2).

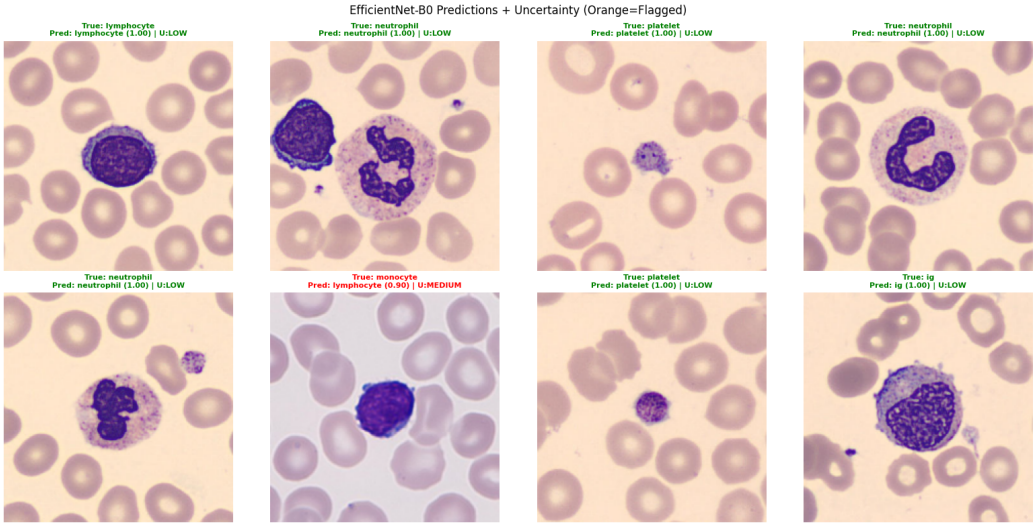


Figure 11: MC-Dropout predictions of the original (v1) model on eight test images. The seven correct predictions receive the uncertainty level LOW; the one error (a monocyte predicted as a lymphocyte with confidence 0.90, red title) receives MEDIUM. MEDIUM is not flagged, so this error would not by itself trigger expert review — a pattern that the systematic evaluation below confirms.

Does the uncertainty estimate work? Table 10 and Figure 12 answer the question left open by the design: whether the uncertainty levels actually identify unreliable predictions. On the in-distribution test set the answer is clearly yes.

Table 10: MC-Dropout uncertainty levels on the 3,419 test images (v2 model, $T = 20$); 49 cells were misclassified in total. Calibration: ECE 0.0039 (single pass) and 0.0032 (MC-Dropout). Error detection: AUROC 0.971 ($1 - \bar{p}_{(1)}$), 0.970 (entropy), 0.968 (variance).

Level	Cells	% of cells	Accuracy	Errors	% of errors
LOW	3,255	95.2	99.72 %	9	18
MEDIUM	98	2.9	84.69 %	15	31
HIGH (flagged)	66	1.9	62.12 %	25	51

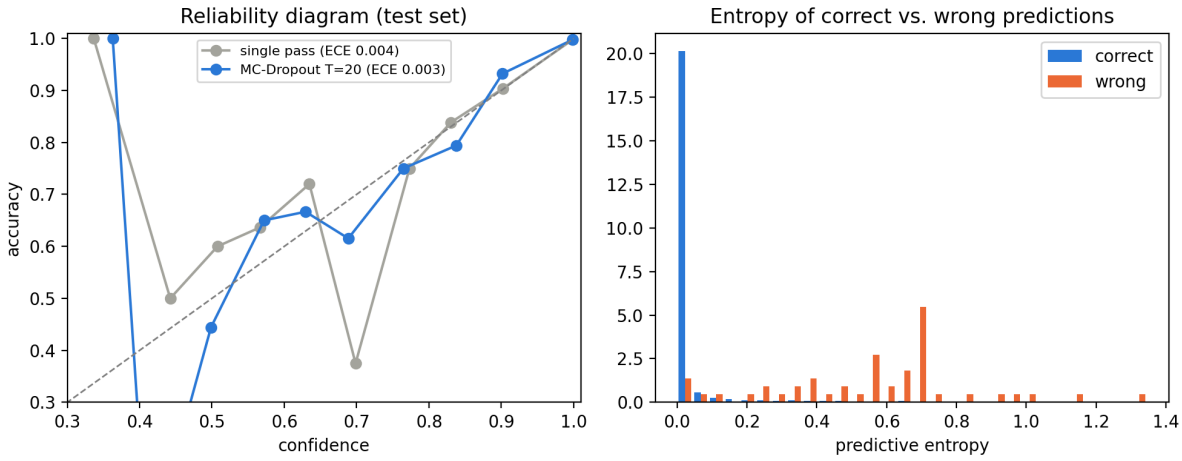


Figure 12: Left: reliability diagram of the v2 classifier on the test set (accuracy per confidence bin; the diagonal is perfect calibration). The low-confidence bins contain very few cells, which explains their erratic course. Right: distribution of predictive entropy for correct and wrong predictions (normalised histograms).

Three observations follow. First, the classifier is *well calibrated* in-domain (ECE below 0.004), and MC-Dropout improves calibration only marginally over a single pass. Second, the uncertainty scores *separate errors from correct predictions very well* (AUROC ≈ 0.97): 95 % of the cells are LOW and 99.7 % of those are correct, whereas the 1.9 % of cells marked HIGH contain half of all errors. Third, the MEDIUM level is a weak spot of the current review rule: its accuracy is only 84.7 %, and its 15 errors reach the expert without a flag. Flagging MEDIUM as well would send 4.8 % of the cells for review and catch 82 % of the errors (40 of 49) — a trade-off between workload and safety that should be set together with clinical users. All of these numbers describe images from the same analyser as the training data; under distribution shift both accuracy and calibration are expected to be worse (see the example below).

An example of domain shift. The PBC images used for training come from a single analyser with standardised staining. Figure 13 shows the combined Stage 1 + Stage 2 output on an image from a different source, produced in the Stage-3 notebook with the original (v1) classifier. The white cell has a segmented nucleus and granular cytoplasm, the appearance of a mature granulocyte, yet the classifier labelled it *erythroblast* with a probability of 0.90. This single example does not measure the size of the problem, but it illustrates why the benchmark accuracy should not be expected to transfer unchanged to smears from other laboratories.

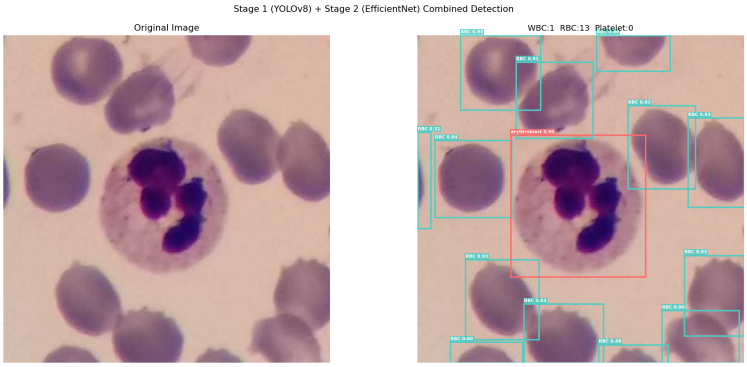


Figure 13: Stage 1 + Stage 2 output on an image from outside the training distribution: 13 red cells and one white cell were detected; the white cell, which appears to be a segmented granulocyte, was classified as *erythroblast* (0.90). (This notebook run used a lower detection threshold than the 0.50 of the final system, so some low-confidence red-cell boxes are shown.)

6.3 Quantitative evaluation of the domain expert

Table 11 and Figure 14 summarise the comparison of the base model, the fine-tuned model and GPT-4o on the corrected, block-level split (Section 3.2); the last paragraph of this section compares them with the first, leaky run.

Table 11: Domain-expert evaluation. In-domain metrics are means over 100 held-out textbook questions; MedMCQA accuracy is over 100 hematology-related exam questions (not run for GPT-4o). + *RAG*: GPT-4o given the five chunks retrieved by the system (full, or cut to 600 characters as in the agent tool). Best value per column in bold.

	In-domain held-out questions				External
System	**Sem. sim.**	**ROUGE-L**	**Source marker**	**Words**	**MedMCQA**
Llama-3.1-8B (base)	0.775	0.170	13 %	159.2	56 %
Llama-3.1-8B + QLoRA (ours)	0.832	**0.455**	99 %	36.5	**61 %**
GPT-4o (reference)	0.792	0.202	88 %	147.4	–
GPT-4o + RAG	**0.848**	0.356	99 %	105.3	–
GPT-4o + RAG (600 characters)	0.831	0.314	96 %	107.1	–
Reference answers	–	–	100 %	34.8	–

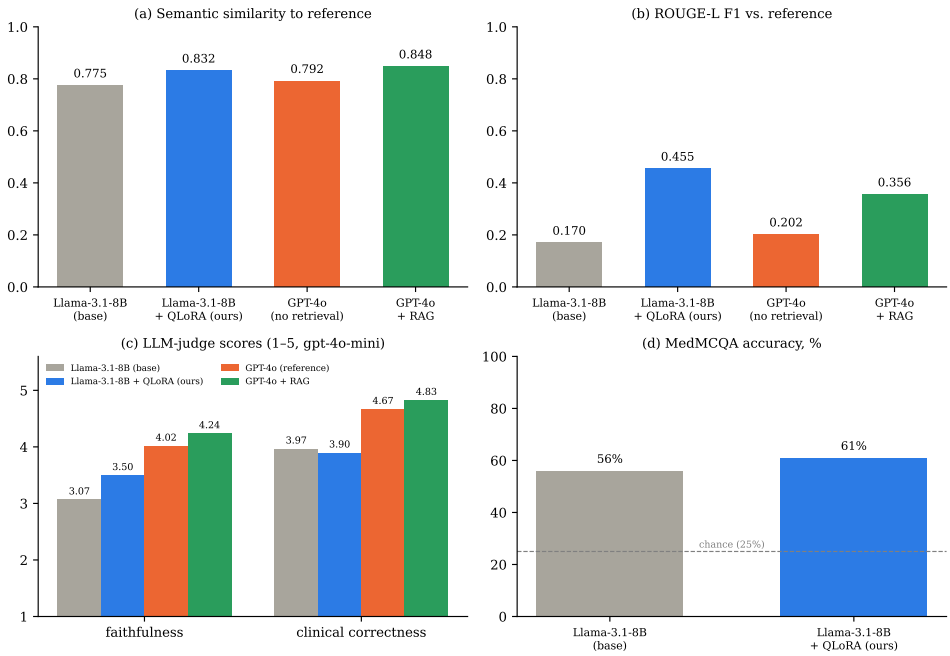


Figure 14: Comparison of the systems: (a) semantic similarity and (b) ROUGE-L F1 against the textbook reference answers on 100 held-out questions; (c) mean LLM-judge scores on the same answers (GPT-4o with and without retrieval); (d) accuracy on 100 hematology-related MedMCQA questions, with the 25 % chance level.

In-domain questions. Fine-tuning brought the model's answers much closer to the textbook reference answers. ROUGE-L rose from 0.170 to 0.455 (+0.285); the paired bootstrap 95 % confidence interval of the improvement is $[0.259,\ 0.313]$, and the fine-tuned model scored higher than the base model on *every one* of the 100 questions. Semantic similarity rose from 0.775 to 0.832. On both measures the fine-tuned 8-billion-parameter model also exceeded GPT-4o (ROUGE-L difference +0.253, 95 % CI $[0.226,\ 0.281]$, higher on 99 of 100 questions). Its answers are about four times shorter than those of the base model and GPT-4o (37 vs. 147–159 words) and close to the length of the reference answers (35 words).

These numbers must be read carefully. The reference answers were written by a GPT model from the textbook passages, in a particular concise style, and the fine-tuned model was trained on 1,898 answers in exactly that style. Part of the gain is therefore a gain in *form*: length, wording and structure. ROUGE-L rewards this strongly; semantic similarity, which is less sensitive to wording, shows a smaller but still clear gain. GPT-4o's lower scores on these two measures do not mean that its answers are less correct — it writes longer, more general explanations that overlap less with the short reference text. The LLM-judge ratings below confirm this.

External questions. On the MedMCQA questions the fine-tuned model answered 61 of 100 correctly and the base model 56. The two models disagreed on 19 questions (base correct only:

7; fine-tuned correct only: 12), and the exact McNemar test gives $p = 0.36$: the difference is well within chance at this sample size. Fine-tuning on 1,898 textbook pairs therefore did not measurably change the model's general hematology knowledge — in particular, it did not damage it. (Both models are well above the 25 % chance level; three of the base model's answers could not be parsed as a letter and count as wrong.)

LLM-judge ratings. Table 12 shows the 500 ratings. Fine-tuning raised *faithfulness* to the textbook answer by 0.43 points (95 % CI [0.25, 0.63]) but left *clinical correctness* unchanged (-0.07, CI $[-0.25, 0.12]$). GPT-4o was rated well above both Llama models, and by a wide margin on correctness (4.67 vs. 3.90 for the fine-tuned model, difference $+0.77$, CI [0.58, 0.96]) — the opposite of the ROUGE-L ranking. The judge's rationales for the fine-tuned model's lowest-rated answers name concrete medical errors, discussed in Section 6.5.

Table 12: LLM-judge scores (GPT-4o-mini, 1–5, mean over 100 held-out questions; standard deviation in brackets) and paired differences.

System	Faithfulness	Clinical correctness
Llama-3.1-8B (base)	3.07 (0.84)	3.97 (0.90)
Llama-3.1-8B + QLoRA (ours)	3.50 (1.18)	3.90 (1.06)
GPT-4o (reference)	4.02 (0.79)	4.67 (0.70)
GPT-4o + RAG	**4.24** (0.64)	**4.83** (0.38)
GPT-4o + RAG (600 characters)	4.14 (0.68)	4.78 (0.42)
Ours − base (95 % CI)	$+0.43$ [0.25, 0.63]	-0.07 $[-0.25, 0.12]$
GPT-4o − ours (95 % CI)	$+0.52$ [0.31, 0.74]	$+0.77$ [0.58, 0.96]
GPT-4o + RAG − GPT-4o (95 % CI)	$+0.22$ [0.03, 0.41]	$+0.16$ [0.02, 0.32]

GPT-4o with retrieval. Given the five passages that the system's own retriever returned, GPT-4o was rated more faithful ($+0.22$, CI [0.03, 0.41]) and more correct ($+0.16$, CI [0.02, 0.32]) than without them, the best of all systems on both scales, and its answers moved closer to the references (ROUGE-L $+0.153$, CI [0.124, 0.183]; semantic similarity 0.848). The gain comes from the 79 questions whose source passage was among the five retrieved: on these, faithfulness rose by $+0.29$ [0.08, 0.52] and correctness by $+0.19$ [0.03, 0.38], whereas on the other 21 neither changed (-0.05 and $+0.05$, both intervals spanning zero). With retrieval, "[Reference *N*]" became a real citation: 89 answers cited at least one passage, and every cited number belonged to a passage that had been given (`rag_analysis.json`). Cutting each passage to 600 characters, as the agent's tool does, lowered ROUGE-L by 0.041 (CI [0.018, 0.065]), while the judge's ratings fell only slightly and not significantly (-0.10 and -0.05). These gains are optimistic, because the questions and reference answers were written from the very chunks

that the retriever finds.

Interpretation. Together, the evidence gives a consistent picture: *QLoRA fine-tuning taught the model how the domain expert should answer — concisely, in the textbook's terminology and with a source line — but it did not teach it new medical knowledge, whereas retrieval made the same hosted model more faithful and more correct.* This agrees with the literature, which finds that retrieval is more effective than fine-tuning for injecting knowledge [9], and it supports the design choice of building the runtime expert around retrieval and tools.

Comparison with the first, leaky run. The first run (`results/domain_expert/`), in which most test questions had a sibling from the same passage in training, gave ROUGE-L 0.161 / 0.433 / 0.181 (base / ours / GPT-4o), a faithfulness gain of +0.54, a correctness gain of +0.05, and a GPT-4o lead over the fine-tuned model of +0.29 (faithfulness) and +0.63 (correctness). The leakage thus did not create the main finding, but it made the fine-tuned model look closer to GPT-4o than it is. The two runs use different test questions, so this comparison is indicative only.

6.4 Retrieval quality of the knowledge base

Table 13 compares the original embedding model with its replacement on the 2,098 questions of the final Q&A data (Section 5.4, part E).

Table 13: Retrieval of the source chunk among 1,049 chunks (2,098 questions; hit rates in %). "Late": 380 questions whose answer lies mainly after the first 150 words of the chunk.

	All questions				hit@5	
Embedding model	**hit@1**	**hit@5**	**+neighbour**	**MRR@10**	**early**	**late**
MiniLM-L6-v2 (256 tokens)	36.3	66.9	77.7	0.495	70.8	49.2
bge-small-v1.5 (512 tokens)	46.2	75.9	84.0	0.589	80.1	56.8
+ query instruction (final)	**46.9**	**77.0**	**84.9**	**0.597**	**81.0**	**58.7**

The 512-token model with the query instruction finds the source chunk in the top five for 77.0 % of the questions instead of 66.9 % (319 vs. 108 discordant questions, exact McNemar $p < 0.001$), so it was adopted. The truncation hypothesis of Section 3.1 is only partly confirmed: both models do much worse on "late" questions, and the new model's gain there (+9.5 points) is no larger than on early ones (+10.2). The improvement thus comes mainly from a stronger retrieval model, not from reading the end of the chunk. Counting the neighbouring chunk raises hit@5 to 84.9 %. The questions were written from the chunks, so the rates are optimistic for real queries (Section 6.8).

6.5 Qualitative analysis: citations and errors

Unsupported source attributions in the base model. No source was given to the models, yet the base Llama model attributed its answer to a specific textbook, author or reference list in **72 of the 100** in-domain answers: the rule of Section 5.4 flagged 68, all of which were confirmed on inspection, and four more were found among the unflagged answers (`flagged_citations_checked.csv`). Typical examples are:

> "According to the textbook *'Hematology: A Very Short Introduction'* by Patricia A. Tanner [1] . . . "
> "References: [1] Lichtman, M. A., Kipps, T. J., & Steinberg, D. A. (2014). Williams Hematology . . . "

In 46 of the 72 answers the named work exists — most often *Harrison's Principles of Internal Medicine* (21 answers) and *Williams Hematology* (12) — but the model had never been given it, and authors, editions or page numbers were often wrong; one answer even contains a verbatim "quotation" with a page number. In 16 answers the named book does not appear to exist: *Hematology: A Very Short Introduction* alone was cited 14 times, attributed to eight different authors. The other 10 are reference-list entries cut off by the answer-length limit. These attributions look convincing but are fabricated — the failure mode described by Walters and Wilder for ChatGPT [3] — and they are triggered here by the system prompt, which asks the model to "cite" a "provided textbook reference" that is not there. Neither the fine-tuned model nor GPT-4o without retrieval named any book or author. GPT-4o without retrieval, however, inserted a bare "[Reference *N*]" marker in 37 answers (the base model in 13), pointing to a reference that did not exist — a milder form of the same behaviour.

Uniform source lines in the fine-tuned model. The fine-tuned model ended 99 of its 100 answers with a source line (Table 11), naming the first textbook in 92 answers and the second in 7. It usually chose the right book — seven of the ten questions from the second book were labelled correctly — which shows that it learned which topics belong to which book. But the line is learned format, not provenance: the model attaches it to correct and incorrect answers alike. Examples of incorrect answers that carry the source line are:

- *Ham test*: the model describes it as a test for haemolytic disease of the newborn; it is the acidified-serum test that confirms paroxysmal nocturnal haemoglobinuria.
- *Low MCV and MCH with normal MCHC*: the model suggests iron deficiency anaemia, whereas the reference answer points to thalassaemia (in iron deficiency the MCHC is also low).

- *Vitamin B12 deficiency*: the model states that serum vitamin B12 is *normal* in megaloblastic anaemia due to B12 deficiency; it is reduced.
- *Platelet count of* 90×10^9*/L* (qualitative question): the model correctly says that this is not always true thrombocytopenia, but claims that platelet clumping makes the count spuriously *raised*; clumping makes it falsely low.

The fine-tuned model therefore looks more authoritative than the base model while being no more knowledgeable. For a domain expert this is an important and somewhat uncomfortable finding: **a citation produced by a language model is not evidence**. A citation is only useful if it points to a passage that was actually retrieved and can be opened. With retrieval this held for every citation of GPT-4o (Section 6.3); in the agent it is enforced by a deterministic check (Section 6.6).

Clinical scenarios. Table 14 summarises the answers to the five hand-written clinical questions. Both models recognise the typical patterns (neutrophilic leukocytosis with toxic granulation suggests infection or inflammation; a microcytic anaemia with target cells suggests a hemoglobinopathy). The fine-tuned model is shorter and more decisive; the base model is longer, lists more possibilities, and often decorates its answer with invented reference numbers.

Table 14: Condensed answers to the five clinical questions (full answers in `evaluation_report.json`).

Question	Base model	Fine-tuned model
78 % neutrophils, toxic granulation, WBC 18×10^9/L	Leukemoid reaction or severe (bacterial) infection; long explanation	"Significant neutrophilic response", suggests an acute inflammatory or infectious process
Only 2 WBCs classified (1 IG, 1 monocyte)	Classification incomplete or inaccurate; the analyser may have missed cells	Calls the differential "abnormal" because only 2 cells were classified; does not say it is uninterpretable
Hb 9.1 g/dL, MCV 68 fL, target cells	Long list: thalassaemias, sickle cell disease, HbC, HbE, iron deficiency, lead poisoning	HbH disease, α-thalassaemia trait, sickle cell trait; globin chain electrophoresis (omits iron deficiency and β-thalassaemia trait)
Myeloblast vs. lymphoblast	Mostly correct list of features, attributed to a textbook it was never given	Brief and broadly correct (prominent nucleolus, few azurophil granules vs. none)
Platelets 90×10^9/L — always true thrombocytopenia?	Attributes the answer to a textbook it was never given, with a wrong author	"No", but says clumping *raises* the count (it lowers it)

Notably, neither model clearly stated that a differential based on two cells cannot be interpreted: the base model called the classification incomplete, and the fine-tuned model called

the differential "abnormal". As the next section shows, the full system made the same kind of mistake when the situation arose in a real run.

6.6 End-to-end case study: an over-interpreted differential

This section analyses one complete run of the system in its default configuration (GPT-4o agent) on the bundled sample image `BCCD_00010.jpg` (640×480 pixels, from the BCCD collection), made on 28 April 2026 with the original (v1) classifier and before the minimum-cell rule existed. It is analysed in detail because it shows both the strengths of the design and a failure that the safety logic did not catch. Its raw output files were later overwritten by a re-run on the same image (see below); the values were recorded from the original output (abridged in Appendix C), and the run's log (`results/case_study_rerun/original_run_20260428_145040.log`) confirms the counts, the 50/50 differential, that no cell was flagged and that review was *not* required. An earlier run with the same configuration that day ended the same way (`original_run_20260428_140721.log`).

What the system did. Stage 1 detected 17 red cells and **2 white cells**. Stage 2 classified one white cell as an immature granulocyte (confidence 0.955, entropy 0.216) and the other as a monocyte (confidence 0.972, entropy 0.146); both received the uncertainty level LOW, so no cell was flagged. The agent then made five tool calls: it looked up the reference ranges of immature granulocytes (0–0.5 %) and monocytes (2–8 %), read the uncertainty summary, and searched the knowledge base for "causes of increased immature granulocytes" and "causes of monocytosis". Its final report stated:

> "The peripheral blood smear shows a significant increase in immature granulocytes (50 %) and monocytes (50 %), which is highly abnormal. [...] Immature granulocytes (IG) at 50 %, significantly above the normal range of 0–0.5 % [Reference 6]." — Recommendation: "Consider bone marrow biopsy if clinically indicated." — `confidence_assessment`: HIGH, `requires_expert_review`: **false**.

What went wrong. Three problems can be identified.

1. **The sample was far too small.** A differential expresses each cell type as a percentage of the white cells counted. Routine manual differentials count at least 100 cells, and the reference method of CLSI H20-A2 counts several hundred [32]. Two cells cannot establish any percentage; "50 % immature granulocytes" is an artefact of dividing by two. The system prompt told the agent that the counts are per microscope field, but it did not tell it to check the number of cells, and nothing in the code did.

2. **A citation was misattributed.** The retrieved chunk labelled Reference 6 is about hypersegmented neutrophils in megaloblastic anaemia. The range 0–0.5 % actually came from the reference-range tool. The claim was correct, but its citation pointed to the wrong evidence — the kind of error that only becomes visible because the tool trace is recorded.

3. **The review flag stayed false.** Both cells had low MC-Dropout uncertainty, so the deterministic rule did not fire, and the model itself set the flag to false while recommending a bone-marrow biopsy. The uncertainty of the *individual cells* was low; what was uncertain was the *differential as a whole*, and the rule did not measure that.

Lessons. The case shows that the design choices of the thesis — a recorded tool trace, retrieved passages that can be checked, uncertainty passed forward — make such failures *detectable*: every step above could be reconstructed from the saved output. But it also shows that per-cell uncertainty is not enough. The fix is simple and deterministic, and it has since been implemented (Section 4.7): whenever fewer than a minimum number of white cells (by default 100, which in practice requires several fields of view) have been classified, the pipeline forces `requires_expert_review` to true and adds an `INSUFFICIENT_CELL_COUNT` flag, and the reasoners are told not to interpret the percentages. Applied to this case (2 cells), the rule sets the flag to true; this is covered by an automated test. When asked about a two-cell differential directly (Table 14), the base model at least called the classification incomplete; in the agent, the structured prompt and the tool outputs led the model to compute and interpret percentages. This is a strong argument for putting such domain rules into code rather than relying on the model's judgement.

Re-run with the corrected system. The same image was analysed again with the corrected (v2) classifier and the minimum-cell rule on 29 September 2026, and twice on 5 November 2026 with the final system (all logs and outputs in `results/case_study_rerun/`); all runs gave the same Stage 1 and Stage 2 results. Detection was unchanged (17 red, 2 white cells). The v2 classifier was far less certain about the first cell (immature granulocyte, confidence 0.42, entropy 1.23, HIGH), while the monocyte stayed confident (0.98, LOW). The pipeline forced `requires_expert_review` to true with `HIGH_UNCERTAINTY` and `INSUFFICIENT_CELL_COUNT` flags, and the agent rated its confidence LOW, although its text still quoted the two 50 % values. The first November run exposed a further gap: the agent answered without calling the knowledge-base tool, yet cited [Reference 1] and [Reference 2], and the interface listed the pipeline's own pre-fetched passages beneath them, so the citations looked supported. The citation check of Section 4.5 was therefore added, and the pre-fetched passages are no longer shown in agent mode. In the final run (Figures 4 and 5) the agent again cited two references without retrieving any, and the check added an `UNSUPPORTED_CITATION` flag. The rules did their job, and the run shows again why such decisions must not be left to the model.

Because BCCD images were recorded with a different microscope and staining than PBC, the two cell labels are themselves uncertain; the HIGH uncertainty of the first cell is consistent with this.

6.7 Discussion of the research questions

RQ1 — blood image classification. Yes, on the benchmark data. Detection reached a mAP@0.50 of 0.985 with perfect white-cell recall, and classification 98.60 % accuracy under a clean train/validation/test protocol (the earlier 99.44 % was 0.84 points too optimistic). Every classified cell carries an uncertainty level and a Grad-CAM map, and the uncertainty is informative: it separates errors from correct predictions with an AUROC of 0.97, and HIGH-uncertainty cells contain half of all errors although they make up only 1.9 % of the cells. How well this transfers to smears from other laboratories was not measured; the single out-of-distribution example in Figure 13 suggests caution.

RQ2 — building the domain expert with RAG. A general LLM was turned into a hematology expert by combining a curated textbook knowledge base, domain tools and a structured output format. The resulting agent produces structured interpretations with a complete, auditable tool trace (Figures 4–5), and its citations are checked against the passages it actually retrieved. Retrieval itself was measured: with a 512-token embedding model, the source passage of a question is among the five retrieved chunks for 77 % of the questions (67 % with the original model), and given these passages GPT-4o answered more faithfully ($+0.22$) and more correctly ($+0.16$) than without them, citing only passages it had been given. The case study shows that this design makes errors visible but does not prevent them; domain rules such as a minimum cell count, and a check that every citation was actually retrieved, must be enforced in code.

RQ3 — domain fine-tuning. QLoRA fine-tuning of Llama-3.1-8B for under an hour on a free GPU produced a model whose answers match textbook reference answers much better than those of the base model and even of GPT-4o (ROUGE-L 0.455 vs. 0.170 and 0.202), and which no longer attributes its answers to textbooks it was never given. The LLM judge agreed: faithfulness to the textbook improved ($+0.43$), clinical correctness did not (-0.07, not significant), and GPT-4o, with or without retrieval, remained clearly more correct. Accuracy on external exam questions did not change significantly (61 % vs. 56 %, $p = 0.36$), and the model learned to attach a source line regardless of correctness. Fine-tuning is therefore suitable for adapting the *behaviour* of an open, self-hostable expert, but it must be combined with retrieval to supply knowledge and real provenance.

RQ4 — safety and auditability. The system passes all safety-relevant information forward and forces expert review when any cell is uncertain. The case study showed the limits of the original rule: it reacted to uncertain *cells*, not to an unreliable *differential*. A minimum-cell rule was therefore added, and later a citation check, after the agent cited passages it had never retrieved. Auditability worked as intended — the misattributed citations and the faulty reasoning could be traced step by step.

6.8 Limitations and threats to validity

- **Synthetic reference answers.** The in-domain test answers were generated by a GPT model from the textbook, not written by hematologists. They may contain errors, and they favour a model trained on the same style. Some questions refer to "the passage", and a few ask about the second book's list of contributors (for example, who heads a university division) rather than about hematology — artefacts of the generation.
- **Data leakage in the first fine-tuning run** was corrected (Section 3.2), but 20 of the 100 test chunks still share a 40-word overlap with a neighbouring training chunk.
- **Small, noisy external test.** The MedMCQA subset has 100 questions, about a quarter of which are only loosely related to hematology; small differences cannot be detected with this sample size.
- **No human ratings.** Correctness was assessed with automatic similarity measures, an LLM judge (GPT-4o-mini) and by hand for a few answers; no clinician evaluated the answers. An LLM judge has its own biases, and one from the same vendor as GPT-4o may favour GPT-4o's style.
- **Training format of the fine-tuned model.** The training examples contain no retrieved passage, so the model was not trained to read and cite evidence. The fine-tuned model was evaluated stand-alone, not inside the agent.
- **Classifier evaluation.** The corrected (v2) split is stratified by class but at image level, not by donor, so cells from the same person can appear in training and test data. The original (v1) evaluation, which selected its checkpoint on the test images, is reported only for comparison. (A mismatch in the inference code, which re-created the second dropout layer with $p = 0.2$ instead of the $p = 0.3$ used in training, was found during this work and corrected; the case-study run predates the correction.)
- **No external validation of the vision stages.** Both vision datasets were recorded under controlled conditions; smears from other microscopes, stains or laboratories may reduce performance substantially.

- **Uncertainty thresholds and review rule** were set by hand. They were evaluated on in-distribution test data (Table 10), where MEDIUM cells turned out to be noticeably less reliable than LOW cells but are not flagged; their behaviour under distribution shift is unknown.
- **Retrieval evaluation on synthetic questions.** The questions were generated from the chunks themselves, so hit rates and the gains of GPT-4o + RAG are optimistic for real queries, and the embedding model was chosen on the same questions, without a separate test set. The RAG condition used one retrieval step, as in the linear mode; the tool-using agent was not evaluated on the question set.
- **Copyright of the knowledge base.** The textbooks are copyrighted and cannot be redistributed, which limits exact reproduction of the knowledge base by others. The synthetic question–answer pairs generated from them, and the adapter trained on them, are published with the project.

7 Conclusion

On the topic of developing a domain expert with a large language model like ChatGPT, this thesis built a hybrid multimodal lab assistant for peripheral blood smears, in which deep-learning models classify the cells in the image and a domain-expert LLM, grounded in hematology textbooks through RAG, interprets the findings. The domain expert was built in two complementary forms. The first is a GPT-4o agent that answers from a retrieval-grounded hematology knowledge base of 1,049 textbook passages, uses domain tools for reference ranges, cell counts and uncertainty, and returns a structured interpretation with a complete tool trace. The second is an open-weight Llama-3.1-8B model, adapted to the domain with QLoRA in under an hour on a free GPU, which can run on local hardware without a commercial API. The knowledge base was checked as well: after its original embedding model was found to ignore the end of almost every passage, it was replaced by one whose retrieval finds the source passage of a question in the top five for 77 % instead of 67 % of the questions, and given these passages GPT-4o answered more faithfully and more correctly than without them. Two image-analysis stages supply the case evidence: a YOLOv8s detector (mAP@0.50 = 0.985) and an EfficientNet-B0 classifier (98.60 % accuracy on a clean held-out test set), whose predictions carry Monte Carlo Dropout uncertainty — which identifies misclassified cells with an AUROC of 0.97 — and Grad-CAM saliency.

The central finding concerns what each technique contributes to a domain expert. Fine-tuning changed *how* the model answers: its answers became concise and textbook-like, matching held-out reference answers far better than those of the base model and of GPT-4o, and it stopped attributing its answers to textbooks it had never been given. It did not change *what* the model knows: an LLM judge found no gain in clinical correctness, accuracy on external exam questions did not change significantly, and the model attached a source line to incorrect answers as confidently as to correct ones. A citation generated by a language model is therefore not evidence. Knowledge and provenance must come from retrieval of real, inspectable passages — given such passages, the same GPT-4o was rated more faithful and more correct, and cited only passages it had received — and the decision to involve a human must be taken by explicit rules. The end-to-end case study confirmed both points: the recorded tool trace made every error traceable, but the expert turned a two-cell sample into a “highly abnormal” differential, and in a later run cited passages it had never retrieved; deterministic rules for both have since been added.

The practical answer to the thesis topic is thus a layered design: *fine-tuning for behaviour, retrieval for knowledge, tools for case data, and deterministic rules for safety*. The system is a research prototype. It has not been validated clinically, its vision models have not been tested on data from other laboratories, and its expert answers were evaluated with automatic measures rather than by hematologists.

7.1 What was not achieved

Several aims were met only partly, and they are stated here explicitly.

- **The fine-tuned open model did not become a better expert in substance.** The hope that QLoRA fine-tuning would make Llama-3.1-8B more knowledgeable was not confirmed: clinical correctness (LLM judge) and exam accuracy were unchanged, and the model remained clearly less correct than GPT-4o. It is therefore not yet a replacement for the hosted model, only a self-hostable starting point.
- **The fine-tuned model was not evaluated inside the full assistant.** It was tested standalone, without retrieved passages; the combination of the open model with RAG and tools — the configuration that the results suggest — was not run end-to-end.
- **The end-to-end evaluation is a single case.** Only one smear image was analysed in depth, in several runs. It exposed real failures, which led to new safety rules, but a systematic evaluation over many smears, ideally rated by hematologists, is still missing.
- **The agent does not reliably use its retrieval tool.** In the final case-study runs it cited references without retrieving any. The new check flags such citations, but it does not make the agent retrieve.
- **Two first evaluations were flawed.** The original classifier run selected its checkpoint on the test images; it was corrected by retraining, and the corrected accuracy (98.60 %) is 0.84 points lower than the original figure. The first fine-tuning run used a train/test split that was effectively random, so most test questions had a sibling from the same passage in training; the data were regenerated with chunk labels, the model was retrained on a block-level split, and the conclusions held, although the fine-tuned model now looks less close to GPT-4o.
- **Generalisation was not measured.** All image results come from curated benchmark data. A single example from another source was misclassified, which suggests that performance on smears from other laboratories may be lower.
- **The optional CBC input was implemented but not evaluated.**

None of these points invalidates the main conclusion; rather, they define the limits within which it holds.

7.2 Future work

1. **Multi-field analysis.** The new minimum-cell rule means that a single field of view will almost always be flagged. Aggregating the cells of many fields (or a whole-slide image)

into one differential is the natural next step towards a clinically meaningful count.

2. **Mandatory retrieval.** Make the agent retrieve before it answers, and pass it full passages: cutting them to 600 characters lowered the overlap of the answers with the references.
3. **Retrieval-aware fine-tuning.** Train the open model on examples that include the retrieved passage, so that it learns to answer from and cite the evidence it is given, and evaluate it inside the agent against GPT-4o on the same cases.
4. **Expert evaluation.** Have hematologists or medical students rate a sample of answers for correctness and faithfulness, and compare their ratings with those of the LLM judge; likewise, test retrieval with questions written independently of the textbook chunks.
5. **Review threshold.** Decide with clinical users whether MEDIUM-uncertainty cells should also be flagged: on the test set this would catch 82 % instead of 51 % of the errors at a review rate of 4.8 % of cells.
6. **External validation.** Test both vision models on smears from another source (e.g. Raabin-WBC or local data), ideally with a donor-level split, and repeat the calibration and error-detection analysis under this distribution shift, comparing MC-Dropout with deep ensembles.
7. **Laboratory values.** Add a CBC input to the web interface and evaluate whether CBC values improve the interpretations.

8 Bibliography

Books, journal articles and conference papers

[1] K. Singhal, S. Azizi, T. Tu, S. S. Mahdavi, J. Wei, H. W. Chung, et al. Large language models encode clinical knowledge. *Nature*, 620:172–180, 2023. URL https://doi.org/10.1038/s41586-023-06291-2.

[2] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, et al. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023. URL https://doi.org/10.1145/3571730.

[3] W. H. Walters and E. I. Wilder. Fabrication and errors in the bibliographic citations generated by ChatGPT. *Scientific Reports*, 13:14045, 2023. URL https://doi.org/10.1038/s41598-023-41032-5.

[4] A. Acevedo, S. Alférez, A. Merino, L. Puigví, and J. Rodellar. Recognition of peripheral blood cell images using convolutional neural networks. *Computer Methods and Programs in Biomedicine*, 180: 105020, 2019. URL https://doi.org/10.1016/j.cmpb.2019.105020.

[5] M. Habibzadeh, M. Jannesari, Z. Rezaei, H. Baharvand, and M. Totonchi. Automatic white blood cell classification using pre-trained deep learning models: ResNet and Inception. In *Tenth International Conference on Machine Vision (ICMV 2017), Proc. SPIE 10696*, 2018. URL https://doi.org/10.1117/12.2311282.

[6] A. Grattafiori, A. Dubey, A. Jauhri, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL https://arxiv.org/abs/2407.21783.

[7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020. URL https://arxiv.org/abs/2005.11401.

[8] N. Reimers and I. Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP*, 2019. URL https://arxiv.org/abs/1908.10084.

[9] O. Ovadia, M. Brief, M. Mishaeli, and O. Elisha. Fine-tuning or retrieval? comparing knowledge injection in LLMs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024. URL https://arxiv.org/abs/2312.05934.

[10] Z. Chen, A. Hernández Cano, A. Romanou, A. Bonnet, K. Matoba, F. Salvi, et al. MEDITRON-70B: Scaling medical pretraining for large language models. *arXiv preprint arXiv:2311.16079*, 2023. URL https://arxiv.org/abs/2311.16079.

[11] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. URL https://arxiv.org/abs/2106.09685.

[12] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023. URL https://arxiv.org/abs/2305.14314.

[13] Y. Wang, Y. Kordi, S. Mishra, A. Liu, N. A. Smith, D. Khashabi, and H. Hajishirzi. Self-Instruct: Aligning language models with self-generated instructions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023. URL https://arxiv.org/abs/2212.10560.

[14] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2210.03629.

[15] A. Pal, L. K. Umapathi, and M. Sankarasubbu. MedMCQA: A large-scale multi-subject multi-choice dataset for medical domain question answering. In *Proceedings of the Conference on Health, Inference, and Learning (CHIL), PMLR 174*, pages 248–260, 2022. URL https://proceedings.mlr.press/v174/pal22a.html.

[16] C.-Y. Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out (ACL Workshop)*, pages 74–81, 2004. URL https://aclanthology.org/W04-1013/.

[17] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*, 2023. URL https://arxiv.org/abs/2306.05685.

[18] A. Acevedo, A. Merino, S. Alférez, Á. Molina, L. Boldú, and J. Rodellar. A dataset of microscopic peripheral blood cell images for development of automatic recognition systems. *Data in Brief*, 30: 105474, 2020. URL https://doi.org/10.1016/j.dib.2020.105474.

[19] L. Gan, X. Li, and X. Wang. A curated and re-annotated peripheral blood cell dataset integrating four public resources. *Scientific Data*, 12:1694, 2025. URL https://doi.org/10.1038/s41597-025-05980-z.

[20] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1321–1330, 2017. URL https://arxiv.org/abs/1706.04599.

[21] B. Lakshminarayanan, A. Pritzel, and C. Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017. URL https://arxiv.org/abs/1612.01474.

[22] Y. Gal and Z. Ghahramani. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, pages 1050–1059, 2016. URL https://arxiv.org/abs/1506.02142.

[23] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra. Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 618–626, 2017. URL https://arxiv.org/abs/1610.02391.

[24] S. M. Kawthalkar. *Essentials of Haematology*. Jaypee Brothers Medical Publishers, New Delhi, 1st edition, 2006. ISBN 81-8061-633-9.

[25] A. H. Schmaier and H. M. Lazarus, editors. *Concise Guide to Hematology*. Wiley-Blackwell, Chichester, 2012.

[26] M. Tan and Q. V. Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 6105–6114, 2019. URL https://arxiv.org/abs/1905.11946.

[27] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020. URL https://arxiv.org/abs/2002.10957.

[28] S. Xiao, Z. Liu, P. Zhang, N. Muennighoff, D. Lian, and J.-Y. Nie. C-Pack: Packed resources for general Chinese embeddings. *arXiv preprint arXiv:2309.07597*, 2023. URL https://arxiv.org/abs/2309.07597.

Standards, laws and official guidance

[29] U.S. Food and Drug Administration, Health Canada, and MHRA. Good machine learning practice for medical device development: Guiding principles, 2021. URL https://www.fda.gov/medical-devices/software-medical-device-samd/good-machine-learning-practice-medical-device-development-guiding-principles.

[30] European Parliament and Council of the European Union. Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act), 2024. URL https://eur-lex.europa.eu/eli/reg/2024/1689/oj.

[31] World Health Organization. Ethics and governance of artificial intelligence for health: Guidance on large multi-modal models, 2024. URL https://www.who.int/publications/i/item/9789240084759.

[32] Clinical and Laboratory Standards Institute. H20-A2: Reference leukocyte (WBC) differential count (proportional) and evaluation of instrumental methods; approved standard – second edition, 2007. URL https://clsi.org/.

Internet sources (software, datasets and web pages)

[33] OpenAI. Hello GPT-4o, 2024. URL https://openai.com/index/hello-gpt-4o/.

[34] LangChain Inc. LangGraph: building stateful, multi-actor applications with LLMs, 2024. URL https://github.com/langchain-ai/langgraph.

[35] G. Jocher, A. Chaurasia, and J. Qiu. Ultralytics YOLOv8, 2023. URL https://github.com/ultralytics/ultralytics.

[36] P. Mooney. BCCD dataset: Blood cell count and detection, 2018. URL https://github.com/Shenggan/BCCD_Dataset.

[37] Chroma. Chroma: the open-source AI-native vector database, 2024. URL https://github.com/chroma-core/chroma.

[38] D. Han and M. Han. Unsloth: faster and more memory-efficient LLM fine-tuning, 2023. URL https://github.com/unslothai/unsloth.

9 Appendix

A. Reproducing the results

Result	Where to find or re-run it
Domain-expert training and evaluation (final run)	`Notebooks/Domain_Expert_Finetune/domain_expert_evaluation_v2.ipynb` (Colab T4, ≈2 h); outputs and executed notebook in `results/domain_expert_v2/`; GPT-4o answers added with `scripts/add_gpt4o_baseline.py`
Q&A dataset generation	`scripts/build_finetune_dataset.py`; data in `data/finetune/train_v2.jsonl`; split check `scripts/check_v2_split.py`
Statistics and figures	`scripts/analyze_domain_expert.py`, `scripts/analyze_judge.py`, `scripts/plot_domain_expert_figures.py`
GPT-4o + RAG on the held-out questions	`scripts/run_rag_baseline.py`, then the judge and `scripts/analyze_rag.py`; outputs `results/domain_expert_v2/rag_retrieval.json`, `rag_analysis.json`
Check of source attributions	`results/domain_expert_v2/flagged_citations_checked.csv`, `citation_check_summary.json`
LLM-judge scores	`scripts/judge_domain_expert.py`; output `results/domain_expert_v2/judge_scores.json`
First (leaky) run and leakage check	`Notebooks/Domain_Expert_Finetune/domain_expert_evaluation.ipynb`, `results/domain_expert/`; `scripts/check_split_leakage.py`
Chunk lengths and retrieval test	`scripts/check_chunk_tokens.py`, `scripts/eval_retrieval.py`; outputs `results/kb_chunk_token_stats.json`, `results/retrieval_eval/`
Published adapter	`https://huggingface.co/Afridi07/hematology-llama-3.1-8b-lora` (uploaded with `Notebooks/Domain_Expert_Finetune/publish_adapter_to_hf.ipynb`)
Detector training and test metrics	`Notebooks/YOLOv8_detection/YOLOv8_on_TBL_PBC_dataset.ipynb`
Classifier (v2), test metrics and uncertainty evaluation	`Notebooks/Efficientnet_classification/effnet_v2_proper_split.ipynb` (Colab T4, ≈1 h); outputs in `results/effnet_v2/`; checkpoint `models/efficientnet_wbc_v2.pt`
Classifier (v1, original)	`Notebooks/Efficientnet_classification/Efficientnet_classification.ipynb`
Case-study runs	all logs and outputs in `results/case_study_rerun/` (see its README); original run: Appendix C
Web-interface figures	`scripts/render_ui_screenshots.py` (renders the saved case-study output with the real frontend)
Running the full system	`python main.py analyze <image>`, or `start.ps1` for the web interface
All runtime settings	`config.yaml`

B. Main configuration values

Key	Value	Meaning
`detection.confidence_threshold`	0.50	minimum box confidence
`classification.mc_dropout_passes`	20	T in MC-Dropout
`classification.uncertainty.low`	conf. ≥ 0.85, entropy < 0.3	LOW level
`classification.uncertainty.medium`	conf. ≥ 0.65, entropy < 0.6	MEDIUM level
`rag.chunking.chunk_size / overlap`	200 / 40 words	text chunking
`rag.embedding.model_name`	`bge-small-en-v1.5`	embedding model
`rag.embedding.query_instruction`	retrieval prefix	added to queries only
`rag.retrieval.top_k`	5	chunks per query
`rag.internet.enabled`	false	web augmentation
`reasoning.mode`	agent	`agent` or `linear`
`reasoning.agent.max_iterations`	6	agent step budget
`llm.model_name / temperature`	gpt-4o / 0.1	language model
`llm.safety.min_wbc_for_differential`	100	minimum-cell rule

C. Output of the domain-expert agent (abridged)

The output of the original case-study run of 28 April 2026 (Section 6.6), shortened:


```
{
 "clinical_interpretation": "The peripheral blood smear shows a significant
    increase in immature granulocytes (50%) and monocytes (50%), which is
    highly abnormal. ...",
 "key_findings": [
    "Immature granulocytes (IG) at 50%, significantly above the normal
     range of 0-0.5% [Reference 6].", "..."],
 "differential_diagnoses": [
    "Infection or bone marrow stress [Reference 6] - ...", "..."],
 "recommendations": [
    "...", "Consider bone marrow biopsy if clinically indicated."],
 "safety_flags": [
    "High levels of immature granulocytes and monocytes warrant further
     investigation ..."],
 "confidence_assessment": "HIGH",
 "requires_expert_review": false,
 "reasoning_mode": "agent",
 "agent_trace": [
  {"type": "tool_call", "name": "lookup_lab_reference_ranges",
   "args": {"parameter": "ig"}},
  {"type": "tool_call", "name": "lookup_lab_reference_ranges",
   "args": {"parameter": "monocyte"}},
  {"type": "tool_call", "name": "get_uncertainty_summary", "args": {}},
  {"type": "tool_call", "name": "query_knowledge_base",
   "args": {"query": "causes of increased immature granulocytes"}},
  {"type": "tool_call", "name": "query_knowledge_base",
   "args": {"query": "causes of monocytosis"}},
  ... five tool results and the final message ...
 ]
}
```


D. Declaration on the use of generative AI

Generative AI tools were used in this work in four ways: (1) GPT-4o and a Llama-3.1 model are components of the system and are described as such; (2) an OpenAI GPT model generated the synthetic question–answer pairs used for fine-tuning and testing (Section 3.2); (3) GPT-4o-mini served as the LLM judge in the evaluation (Section 5.4); and (4) AI assistants, including Anthropic's Claude, were used to help draft and refactor code and text, re-plot figures and render the interface screenshots, run checks, and review the work for errors and inconsistencies. All

design decisions, experiments, reported results and final wording are the author's responsibility, and all numbers in this thesis were taken from the saved outputs of the experiments.

Acknowledgment

I would like to thank my supervisor, Prof. Dr. András Hajdu, for his guidance and patience throughout this project. I am grateful to the authors of the TXL-PBC, PBC and MedMCQA datasets for making their data openly available, and to the maintainers of Ultralytics, PyTorch, `timm`, Unsloth, Hugging Face Transformers, ChromaDB, LangChain/LangGraph, FastAPI and Vite, whose open-source work made this single-author project possible.